

Java Collections - Complete Solution Reference

Java Collections — Complete Solution Reference

All 100 Problems | List · Map · Queue · Set

Table of Contents

How to Use This Reference

Java Collections — Solution Reference

Module 1: List

Problem 1 — Create and Iterate an ArrayList

Problem 2 — Insert and Remove at Specific Index

Problem 3 — Size, Clear, and isEmpty

Problem 4 — First, Last, and Middle Element

Problem 5 — Sum and Average

Problem 6 — contains() and indexOf()

Problem 7 — Reverse Without Collections.reverse()

Problem 8 — Sort and Identify Extremes

Problem 9 — Remove Duplicates, Preserve Order

Problem 10 — Frequency Count

Problem 11 — Merge Two Sorted Lists

Problem 12 — Stack Simulation via ArrayList: Bracket Validation

Problem 13 — Left-Rotate by K Positions

Problem 14 — Custom Multi-Field Sort

Problem 15 — Subarray with Target Sum (Sliding Window)

Problem 16 — Next Greater Element (Monotonic Stack)

Problem 17 — Longest Increasing Subsequence (Length Only)

Problem 18 — Kadane's Algorithm with Indices

Problem 19 — Merge Overlapping Intervals

Problem 20 — Three Sum

Problem 21 — Alternate Positive and Negative

Problem 22 — List Intersection with Duplicates

Problem 23 — Best Time to Buy and Sell Stock

Problem 24 — Minimum Jumps to Reach Last Index

Java Collections — Solution Reference

Module 2: Map

Problem 25 — Create and Iterate a HashMap

Problem 26 — Get, containsKey, Remove

Problem 27 — Character Frequency (Manual)

Problem 28 — Phone Directory

Problem 29 — Anagram Check (No Sorting)

Problem 30 — First Non-Repeating Character (LinkedHashMap)

Problem 31 — Word Frequency, Sorted Descending

Problem 32 — Two Sum

Problem 33 — Group Anagrams

Problem 34 — Subarray Sum Equals K

Problem 35 — Longest Consecutive Sequence

Problem 36 — LRU Cache (LinkedHashMap)

Problem 37 — Top K Frequent Elements

Problem 38 — Isomorphic Strings

Problem 39 — Minimum Window Substring

Problem 40 — HashMap from Scratch (Separate Chaining)

Problem 41 — Course Schedule (BFS Topological Sort)

Problem 42 — Longest Substring Without Repeating Characters

Problem 43 — Longest Subarray with Equal 0s and 1s

- Problem 44 — Word Pattern Matching (Bijection)
- Problem 45 — Meeting Rooms (Minimum Rooms)
- Problem 46 — Group Logs by ID, Sort by Timestamp
- Problem 47 — Single Number (XOR vs HashMap)
- Problem 48 — Design RandomizedSet (O(1) Operations)
- Problem 49 — Subarray Product Less Than K
- Problem 50 — Range Sum Query (Prefix Sum Array)

Java Collections — Solution Reference

Module 3: Queue

- Problem 51 — Basic Queue Operations
- Problem 52 — Peek vs Poll
- Problem 53 — Reverse a Queue Using a Stack
- Problem 54 — Print Queue Without Data Loss
- Problem 55 — Queue Implemented Using Two Stacks
- Problem 56 — Stack Implemented Using Two Queues
- Problem 57 — Level Order Tree Traversal
- Problem 58 — First Non-Repeating Character in a Stream
- Problem 59 — Circular Queue
- Problem 60 — Sliding Window Maximum (Monotonic Deque)
- Problem 61 — Shortest Path in Grid Avoiding Walls (BFS)
- Problem 62 — Rotten Oranges (Multi-Source BFS)
- Problem 63 — Task Scheduler (Cooldown with PriorityQueue)
- Problem 64 — Reconstruct Queue by Height
- Problem 65 — Kth Largest Element in a Stream
- Problem 66 — Merge K Sorted Lists Using PriorityQueue
- Problem 67 — Implement Deque from Scratch (Doubly Linked List)
- Problem 68 — Sliding Window Maximum: Brute Force vs Optimal
- Problem 69 — Connect Nodes at Same Level (Next Right Pointer)
- Problem 70 — Shortest Path in Maze (BFS, Four Directions)
- Problem 71 — Running Median (Two Heaps)
- Problem 72 — Job Scheduling for Maximum Profit
- Problem 73 — Snake and Ladder (Minimum Dice Rolls via BFS)
- Problem 74 — Word Ladder: All Shortest Transformation Sequences

Java Collections — Solution Reference

Module 4: Set

- Problem 75 — Create a HashSet (No Duplicates)
- Problem 76 — Contains and Remove
- Problem 77 — HashSet vs LinkedHashSet vs TreeSet
- Problem 78 — Deduplicate List, Preserve Order
- Problem 79 — Union (Manual, Without addAll)
- Problem 80 — Intersection (Manual)
- Problem 81 — Difference (Manual, then Verified)
- Problem 82 — Subset Check
- Problem 83 — Distinct Words, Sorted
- Problem 84 — Longest Consecutive Sequence
- Problem 85 — All Pairs with a Given Difference
- Problem 86 — Custom Comparable for TreeSet Ordering
- Problem 87 — Happy Number (Cycle Detection)
- Problem 88 — Word Ladder: Shortest Path Length (BFS)
- Problem 89 — Number of Islands (BFS with Visited Set)
- Problem 90 — Reconstruct Itinerary (DFS with PriorityQueue)
- Problem 91 — Design Twitter (Map + Set + PriorityQueue)
- Problem 92 — Longest Substring with At Most K Distinct Characters
- Problem 93 — Divide into K Consecutive Groups
- Problem 94 — K Closest Points to the Origin
- Problem 95 — Cycle Detection in a Directed Graph (DFS with Recursion Stack)

Problem 96 — Longest Word Buildable One Character at a Time
Problem 97 — Four Sum
Problem 98 — Accounts Merge (Union-Find with Email Mapping)
Problem 99 — All Distinct Palindromic Substrings
Problem 100 — Distinct Subsequences Count
Document Summary

Java Collections — Complete Solution Reference

All 100 Problems | List · Map · Queue · Set

Repository: SuperFaculties — DSA Training Materials **Audience:** Faculty Development Program (FDP) — SAGE University **Course:** Java DSA Training | AlgoTutor Program **Prepared by:** Nidhish Raj Mourya
Contact: +91-6265883747

Table of Contents

Module	Topics Covered	Problems
<u>Module 1:</u> <u>List</u>	ArrayList, two-pointer, sliding window, Kadane's, LIS	1-24
<u>Module 2:</u> <u>Map</u>	HashMap, LinkedHashMap, TreeMap, LRU cache, topological sort	25-50
<u>Module 3:</u> <u>Queue</u>	LinkedList, ArrayDeque, PriorityQueue, BFS, heaps	51-74
<u>Module 4:</u> <u>Set</u>	HashSet, LinkedHashSet, TreeSet, Union-Find, graph cycles	75-100

How to Use This Reference

Each problem entry includes: 1. **Problem statement** — the exact requirement as assigned 2. **Solution code** — complete, compilable Java implementation 3. **Explanation** — the underlying approach, key design decisions, and complexity analysis 4. **Teaching notes** (where applicable) — common student misconceptions and discussion points for classroom delivery

This document is intended as a faculty reference for delivering the Java Collections module of the DSA training curriculum. Problems progress from foundational API usage to interview-level algorithmic patterns within each module.

Java Collections — Solution

Reference

Module 1: List

(ArrayList)

Repository: SuperFaculties — DSA Training Materials **Audience:** Faculty Development Program (FDP) — SAGE University **Prepared by:** Nidhish Raj Mourya

Problem 1 — Create and Iterate an ArrayList

Statement: Create an ArrayList of 5 city names and print each city on a new line using a for-each loop.

```
import java.util.ArrayList;

public class Solution1 {
    public static void main(String[] args) {
        ArrayList<String> cities = new ArrayList<>();
        cities.add("Delhi");
        cities.add("Mumbai");
        cities.add("Indore");
        cities.add("Pune");
        cities.add("Chennai");

        for (String city : cities) {
            System.out.println(city);
        }
    }
}
```

Explanation: The `ArrayList<String>` declaration creates a resizable array backed collection. The `add()` method appends to the end in $O(1)$ amortized time. The for-each loop uses Java's Iterator internally — the programmer does not manage index bounds manually.

Teaching note: For-each loops are read-safe but not write-safe. Adding or removing elements from the list during iteration throws `ConcurrentModificationException`. This is a common interview question and a frequent source of runtime bugs for beginners.

Problem 2 — Insert and Remove at Specific Index

Statement: Create a list of 5 integers. Add 99 at index 2, then remove the element at index 0. Print the final list.

```
import java.util.ArrayList;

public class Solution2 {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        nums.add(10); nums.add(20); nums.add(30); nums.add(40);
        nums.add(50);

        nums.add(2, 99);
    }
}
```

```

        nums.remove(0);

        System.out.println(nums);    // [20, 99, 30, 40, 50]
    }
}

```

Explanation: `add(index, element)` is an overloaded method — `add(99)` appends to the end, while `add(2, 99)` inserts at a specific position, shifting subsequent elements right.

Common pitfall: `remove(int index)` and `remove(Object obj)` are both valid overloads. Calling `nums.remove(0)` removes the element **at index 0**. If an Integer object with value 0 is passed instead, Java resolves to the object-removal overload, deleting the value 0 if present — a frequent source of confusion.

Internal mechanism: `ArrayList` is array-backed, so insertion or deletion at an arbitrary index is $O(n)$, since all subsequent elements must shift.

Problem 3 — Size, Clear, and isEmpty

Statement: Create an empty `ArrayList`. Add 3 student names. Print the size. Remove all elements. Print whether the list is empty.

```

import java.util.ArrayList;

public class Solution3 {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();
        students.add("Aman");
        students.add("Priya");
        students.add("Rohan");

        System.out.println("Size: " + students.size());

        students.clear();

        System.out.println("Is Empty: " + students.isEmpty());
    }
}

```

Explanation: `size()` runs in $O(1)$ because `ArrayList` internally maintains a counter rather than computing length on demand. `clear()` nulls out internal references for garbage collection but does not necessarily shrink the underlying array's capacity — call `trimToSize()` explicitly if memory reclamation is required. `isEmpty()` is functionally equivalent to `size() == 0` but improves readability.

Problem 4 — First, Last, and Middle Element

Statement: Given a list of fruits, print the first, last, and middle element. Handle both odd and even length lists.

```

import java.util.ArrayList;

public class Solution4 {
    public static void main(String[] args) {

```

```

        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple"); fruits.add("Banana");
    fruits.add("Mango");
        fruits.add("Grapes"); fruits.add("Kiwi");

    int n = fruits.size();
    System.out.println("First: " + fruits.get(0));
    System.out.println("Last: " + fruits.get(n - 1));
    System.out.println("Middle: " + fruits.get(n / 2));
    }
}

```

Explanation: For odd-length lists (size = 5), integer division $5/2 = 2$ correctly identifies the middle index. For even-length lists (size = 4), $4/2 = 2$ returns the upper-middle element; if a lower-middle convention is preferred, use $(n-1)/2$ instead.

Teaching note: “Middle element” is inherently ambiguous for even-length collections. Faculty should establish and communicate a consistent convention before presenting this problem to students.

Problem 5 — Sum and Average

Statement: Create an ArrayList of 10 integers. Compute the sum and average. Print both.

```

import java.util.ArrayList;

public class Solution5 {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        for (int i = 1; i <= 10; i++) nums.add(i);

        int sum = 0;
        for (int n : nums) sum += n;

        double avg = (double) sum / nums.size();

        System.out.println("Sum: " + sum);
        System.out.printf("Average: %.2f\n", avg);
    }
}

```

Explanation: This problem highlights one of Java’s most common type-coercion pitfalls:

```

        double avg = sum / nums.size();           // WRONG – int / int = int
        division, truncates decimal
        double avg = (double) sum / nums.size(); // CORRECT – forces
        floating-point division

```

Without the explicit cast, $55 / 10$ evaluates to 5 (integer division), discarding the fractional component. This should be emphasized as a recurring theme across numeric Java problems.

Problem 6 — contains() and indexOf()

Statement: Given a list of employee names, check if “Aman” exists using `contains()`. Find its index using `indexOf()`. Print -1 if not found.

```
import java.util.ArrayList;

public class Solution6 {
    public static void main(String[] args) {
        ArrayList<String> employees = new ArrayList<>();
        employees.add("Priya"); employees.add("Aman");
        employees.add("Raj"); employees.add("Neha");

        boolean found = employees.contains("Aman");
        System.out.println("Found: " + found);

        int idx = employees.indexOf("Aman");
        System.out.println("Index: " + idx);

        int notFound = employees.indexOf("Sonia");
        System.out.println("Index: " + notFound);    // -1
    }
}
```

Explanation: `contains()` internally invokes `equals()` against each element in sequence — an $O(n)$ linear scan. `indexOf()` already returns -1 when an element is not found; no exception handling is required for this case. This differs from `List.get()`, which throws `IndexOutOfBoundsException` for invalid indices.

Problem 7 — Reverse Without `Collections.reverse()`

Statement: Create a list of integers. Reverse it in-place without using `Collections.reverse()`. Print before and after.

```
import java.util.ArrayList;

public class Solution7 {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        nums.add(5); nums.add(3); nums.add(8); nums.add(1);
        nums.add(9); nums.add(2);

        System.out.println("Before: " + nums);

        int left = 0, right = nums.size() - 1;
        while (left < right) {
            int temp = nums.get(left);
            nums.set(left, nums.get(right));
            nums.set(right, temp);
            left++;
            right--;
        }

        System.out.println("After: " + nums);
    }
}
```

Explanation: This is the canonical **two-pointer** pattern. Two indices converge from opposite ends of the list, swapping elements at each step. `set(index, value)` replaces an element at a given position in

$O(1)$. A temporary variable is required during the swap, since overwriting left before reading its original value would corrupt the operation.

Complexity: $O(n)$ time, $O(1)$ space — true in-place reversal with no auxiliary list allocation.

Problem 8 — Sort and Identify Extremes

Statement: Given a list of company names, sort alphabetically. Print the sorted list along with the lexicographically first and last company.

```
import java.util.ArrayList;
import java.util.Collections;

public class Solution8 {
    public static void main(String[] args) {
        ArrayList<String> companies = new ArrayList<>();
        companies.add("Wipro"); companies.add("TCS");
        companies.add("Infosys"); companies.add("HCL");
        companies.add("Accenture");

        Collections.sort(companies);

        System.out.println("Sorted: " + companies);
        System.out.println("First: " + companies.get(0));
        System.out.println("Last: " + companies.get(companies.size()
- 1));
    }
}
```

Explanation: `Collections.sort()` relies on the natural ordering defined by `String`'s `compareTo()` method, which performs lexicographic comparison based on Unicode character values. Internally, Java uses a variant of **TimSort** — $O(n \log n)$, and stable. Custom objects require implementing `Comparable` or supplying an explicit `Comparator`, since this ordering is not automatically defined.

Problem 9 — Remove Duplicates, Preserve Order

Statement: Create a list with duplicate integers. Remove all duplicates while maintaining insertion order without directly using a Set in the output.

```
import java.util.ArrayList;
import java.util.LinkedHashSet;

public class Solution9 {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        int[] input = {1, 2, 2, 3, 4, 4, 5, 1};
        for (int n : input) nums.add(n);

        LinkedHashSet<Integer> seen = new LinkedHashSet<>(nums);
        ArrayList<Integer> result = new ArrayList<>(seen);

        System.out.println(result);    // [1, 2, 3, 4, 5]
    }
}
```



```
}
```

Explanation: `LinkedHashSet` preserves insertion order while enforcing uniqueness — a plain `HashSet` provides no ordering guarantee.

Teaching note: When duplicate removal must preserve original ordering, use `LinkedHashSet`, never a plain `HashSet`. This distinction is a recurring point of confusion and worth emphasizing explicitly during instruction.

Problem 10 — Frequency Count

Statement: Given a list of integers, count how many times each unique element appears and print the frequency of each.

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;

public class Solution10 {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        int[] input = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5};
        for (int n : input) nums.add(n);

        Map<Integer, Integer> freq = new LinkedHashMap<>();
        for (int n : nums) {
            freq.put(n, freq.getOrDefault(n, 0) + 1);
        }

        for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
            System.out.println(entry.getKey() + " -> " +
entry.getValue());
        }
    }
}
```

Explanation: `getOrDefault(key, defaultValue)` is the idiomatic Java 8+ shorthand for conditional initialization, replacing the more verbose:

```
if (freq.containsKey(n)) {
    freq.put(n, freq.get(n) + 1);
} else {
    freq.put(n, 1);
}
```

`LinkedHashMap` is used here specifically to preserve insertion order in the output. `entrySet()` returns key-value pairs together, which is more efficient than iterating `keySet()` and calling `get()` separately for each key.

Problem 11 — Merge Two Sorted Lists

Statement: Given two sorted `ArrayLists`, merge them into one sorted list without calling `sort()` on the merged result.

```
import java.util.ArrayList;
```

```

public class Solution11 {
    public static void main(String[] args) {
        ArrayList<Integer> a = new ArrayList<>(java.util.List.of(1,
3, 5, 7));
        ArrayList<Integer> b = new ArrayList<>(java.util.List.of(2,
4, 6, 8));

        System.out.println(mergeSorted(a, b));
    }

    static ArrayList<Integer> mergeSorted(ArrayList<Integer> a,
ArrayList<Integer> b) {
        ArrayList<Integer> result = new ArrayList<>();
        int i = 0, j = 0;

        while (i < a.size() && j < b.size()) {
            if (a.get(i) <= b.get(j)) result.add(a.get(i++));
            else result.add(b.get(j++));
        }
        while (i < a.size()) result.add(a.get(i++));
        while (j < b.size()) result.add(b.get(j++));

        return result;
    }
}

```

Explanation: This is the **two-pointer merge** step from merge sort. Since both input lists are pre-sorted, comparing front elements and advancing the smaller pointer produces a sorted result in $O(n + m)$ without re-sorting. The trailing while loops handle the case where one list is exhausted before the other — remaining elements are already sorted and can be appended directly.

Problem 12 — Stack Simulation via ArrayList: Bracket Validation

Statement: Simulate a Stack using ArrayList. Implement push, pop, peek, and isEmpty. Use it to check if a bracket sequence is balanced.

```

import java.util.ArrayList;

public class Solution12 {
    public static void main(String[] args) {
        System.out.println(isBalanced("{[]}")); // true
        System.out.println(isBalanced("{[]}")); // false
    }

    static boolean isBalanced(String s) {
        ArrayList<Character> stack = new ArrayList<>();

        for (char ch : s.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.add(ch);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.remove(stack.size() - 1);
                if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '[')) return false;
            }
        }
        return stack.isEmpty();
    }
}

```

```

        (ch == ']' && top != '[')) return false;
    }
}
return stack.isEmpty();
}
}

```

Explanation: ArrayList can simulate Stack semantics: add() for push, remove(size()-1) for pop, get(size()-1) for peek.

Teaching note: ArrayList is used here for instructional purposes. In production code, ArrayDeque<Character> is preferred, offering dedicated, more readable push()/pop() methods. The stack must be empty at the end of processing — any unmatched opening bracket will remain on the stack.

Problem 13 — Left-Rotate by K Positions

Statement: Given a list of integers and K, left-rotate the list by K positions. Handle K larger than the list size.

```

import java.util.ArrayList;
import java.util.List;

public class Solution13 {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>(List.of(1, 2, 3,
4, 5));

        System.out.println(leftRotate(nums, 2)); // [3,4,5,1,2]
        System.out.println(leftRotate(nums, 7)); // same as k=2
    }

    static ArrayList<Integer> leftRotate(ArrayList<Integer> nums,
int k) {
        int n = nums.size();
        k = k % n;

        ArrayList<Integer> result = new ArrayList<>();
        result.addAll(nums.subList(k, n));
        result.addAll(nums.subList(0, k));

        return result;
    }
}

```

Explanation: The modulo operation $k \% n$ is essential: rotating by 7 positions in a 5-element list is equivalent to rotating by $7 \% 5 = 2$, since a full rotation returns the list to its original order.

subList(start, end) returns a **view** of the original list rather than a copy. If the original list were modified afterward, the subList view would reflect those changes. Copying into a new list via addAll() avoids this aliasing concern.

Problem 14 — Custom Multi-Field Sort

Statement: Create a list of Student objects with name and CGPA. Sort by CGPA descending; if CGPA is equal, sort by name alphabetically.

```
import java.util.ArrayList;
import java.util.List;

public class Solution14 {
    record Student(String name, double cgpa) {}

    public static void main(String[] args) {
        ArrayList<Student> students = new ArrayList<>(List.of(
            new Student("Aman", 8.5), new Student("Priya", 9.0),
            new Student("Raj", 8.5), new Student("Neha", 9.0)
        ));

        students.sort((a, b) -> {
            if (a.cgpa() != b.cgpa()) return
Double.compare(b.cgpa(), a.cgpa());
            return a.name().compareTo(b.name());
        });

        for (Student s : students) System.out.println(s.name() + " - "
+ s.cgpa());
    }
}
```

Explanation: record (Java 14+) generates an immutable data class with accessor methods in a single line. The lambda comparator demonstrates **multi-level sorting**: the primary criterion (CGPA) is checked first; the secondary criterion (name) is evaluated only on ties.

Double.compare(b, a) produces descending order by reversing argument order; Double.compare(a, b) would produce ascending order.

Problem 15 — Subarray with Target Sum (Sliding Window)

Statement: Given a list of integers, find the contiguous subarray that sums to a given target S. Print the start and end indices. Use O(n) sliding window.

```
public class Solution15 {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 7, 5};
        int[] result = subarraySum(arr, 12);
        System.out.println("Indices: " + result[0] + " to " +
result[1]);
    }

    static int[] subarraySum(int[] arr, int target) {
        int left = 0, sum = 0;

        for (int right = 0; right < arr.length; right++) {
            sum += arr[right];

            while (sum > target && left <= right) {
                sum -= arr[left];
                left++;
            }
        }
    }
}
```

```

    }

    if (sum == target) return new int[]{left, right};
}

return new int[]{-1, -1};
}
}

```

Explanation: This is a classic **sliding window** problem, valid only for non-negative integers — negative values invalidate the window-shrinking logic, since increasing the window size does not monotonically increase the sum.

The right pointer expands the window; the left pointer contracts it whenever the running sum exceeds the target.

Complexity: $O(n)$ — each element is added at most once and removed at most once.

Problem 16 — Next Greater Element (Monotonic Stack)

Statement: For each element in a list, find the next greater element to its right. Print -1 if none exists. Use a Stack for $O(n)$ solution.

```

import java.util.ArrayDeque;
import java.util.Deque;

public class Solution16 {
    public static void main(String[] args) {
        int[] arr = {4, 5, 2, 10, 8};
        int[] result = nextGreater(arr);
        for (int x : result) System.out.print(x + " ");
    }

    static int[] nextGreater(int[] arr) {
        int n = arr.length;
        int[] result = new int[n];
        java.util.Arrays.fill(result, -1);

        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
                result[stack.pop()] = arr[i];
            }
            stack.push(i);
        }

        return result;
    }
}

```

Explanation: This is the **monotonic stack** pattern — the stack maintains indices in decreasing value order. When a new element exceeds the value at the stack's top, that top element has found its “next greater” and is popped with its result recorded.

Indices remaining on the stack at the end have no next-greater element and retain the default value -1.

Complexity: $O(n)$ — a substantial improvement over the $O(n^2)$ brute-force approach that scans rightward from every element.

Problem 17 — Longest Increasing Subsequence (Length Only)

Statement: Find the length of the longest strictly increasing subsequence in an ArrayList.

```
import java.util.ArrayList;

public class Solution17 {
    public static void main(String[] args) {
        int[] arr = {10, 9, 2, 5, 3, 7, 101, 18};
        System.out.println(lisLength(arr)); // 4
    }

    static int lisLength(int[] arr) {
        ArrayList<Integer> tails = new ArrayList<>();

        for (int x : arr) {
            int pos = binarySearch(tails, x);
            if (pos == tails.size()) tails.add(x);
            else tails.set(pos, x);
        }

        return tails.size();
    }

    static int binarySearch(ArrayList<Integer> tails, int x) {
        int lo = 0, hi = tails.size();
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (tails.get(mid) < x) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }
}
```

Explanation: This achieves $O(n \log n)$, a substantial improvement over the $O(n^2)$ dynamic programming approach.

`tails[i]` represents the smallest possible tail value for an increasing subsequence of length $i+1$ observed so far. Binary search determines where a new element fits: if it extends beyond the current tails array, the LIS length increases; if it fits within an existing position, that tail is updated to preserve future extension potential.

Important: `tails` is not itself the LIS — it is an auxiliary structure for tracking achievable lengths.

Problem 18 — Kadane's Algorithm with Indices

Statement: Find the maximum sum contiguous subarray in a list that may contain negative integers. Print the max sum and the subarray indices.

```
public class Solution18 {
    public static void main(String[] args) {
        int[] arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
        int[] result = kadane(arr);
        System.out.println("Max Sum: " + result[0] + ", Indices: " +
            result[1] + " to " + result[2]);
    }

    static int[] kadane(int[] arr) {
        int maxSum = arr[0], currSum = arr[0];
        int start = 0, end = 0, tempStart = 0;

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > currSum + arr[i]) {
                currSum = arr[i];
                tempStart = i;
            } else {
                currSum += arr[i];
            }

            if (currSum > maxSum) {
                maxSum = currSum;
                start = tempStart;
                end = i;
            }
        }

        return new int[]{maxSum, start, end};
    }
}
```

Explanation: This is a greedy/dynamic-programming hybrid. At each position, the algorithm decides: extend the current subarray, or restart from this position? If $\text{arr}[i] > \text{currSum} + \text{arr}[i]$, the previous subarray is net-negative and dragging the total down — restarting is better. `tempStart` tracks the potential new starting index; `start/end` record the confirmed best subarray boundaries.

Complexity: $O(n)$ single pass — effectively a rolling simplification of dynamic programming, requiring only the current state rather than a full DP table.

Problem 19 — Merge Overlapping Intervals

Statement: Given a list of intervals represented as start-end pairs, merge all overlapping intervals and return the result.

```
import java.util.*;

public class Solution19 {
    public static void main(String[] args) {
        int[][] intervals = {{1,3},{2,6},{8,10},{15,18}};
        List<int[]> result = mergeIntervals(intervals);
        for (int[] interval : result)
            System.out.println(Arrays.toString(interval));
    }
}
```

```

    }

    static List<int[]> mergeIntervals(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        List<int[]> merged = new ArrayList<>();
        merged.add(intervals[0]);

        for (int i = 1; i < intervals.length; i++) {
            int[] last = merged.get(merged.size() - 1);
            if (intervals[i][0] <= last[1]) {
                last[1] = Math.max(last[1], intervals[i][1]);
            } else {
                merged.add(intervals[i]);
            }
        }

        return merged;
    }
}

```

Explanation: Sorting by start time is a required first step — it ensures overlap detection is sequential and does not require scanning the full list for each interval.

Each new interval need only be compared against the **most recently merged interval**: if its start is less than or equal to that interval's end, it overlaps and the end boundary is extended (taking the maximum). Otherwise, it begins a new, non-overlapping interval.

Problem 20 — Three Sum

Statement: Given a list of integers, find all unique triplets that sum to zero. Avoid duplicate triplets in the output.

```

import java.util.*;

public class Solution20 {
    public static void main(String[] args) {
        int[] nums = {-1, 0, 1, 2, -1, -4};
        System.out.println(threeSum(nums));
    }

    static List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < nums.length - 2; i++) {
            if (i > 0 && nums[i] == nums[i-1]) continue;

            int left = i + 1, right = nums.length - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left],
nums[right]));
                    while (left < right && nums[left] ==
nums[left+1]) left++;

```



```

        while (left < right && nums[right] ==
nums[right-1]) right--;
        left++; right--;
    } else if (sum < 0) left++;
    else right--;
    }
}

return result;
}
}

```

Explanation: This combines sorting with the two-pointer technique. Sorting enables both duplicate detection and the two-pointer search. For each fixed index i , the algorithm searches the remainder of the array with left/right pointers for a pair summing to $-nums[i]$.

Duplicate skipping is critical at three levels: the outer loop (if $(i > 0 \ \&\& \ nums[i] == \ nums[i-1])$ continue;), and both left and right after a match is found, to avoid emitting the same triplet multiple times.

Problem 21 — Alternate Positive and Negative

Statement: Given a list of integers, rearrange it so that positive and negative numbers alternate. If counts are unequal, append the remaining elements at the end.

```

import java.util.ArrayList;
import java.util.List;

public class Solution21 {
    public static void main(String[] args) {
        int[] arr = {1, -2, 3, -4, 5, -6, 7};
        System.out.println(alternate(arr));
    }

    static List<Integer> alternate(int[] arr) {
        List<Integer> pos = new ArrayList<>(), neg = new ArrayList<>
();

        for (int x : arr) {
            if (x >= 0) pos.add(x);
            else neg.add(x);
        }

        List<Integer> result = new ArrayList<>();
        int i = 0, j = 0;
        while (i < pos.size() && j < neg.size()) {
            result.add(pos.get(i++));
            result.add(neg.get(j++));
        }
        while (i < pos.size()) result.add(pos.get(i++));
        while (j < neg.size()) result.add(neg.get(j++));

        return result;
    }
}

```

Explanation: Positives and negatives are first partitioned into separate lists, simplifying the subsequent merge logic. Elements are then drawn alternately from each list — structurally identical to the merge step in merge sort. If one list is exhausted before the other, remaining elements are appended directly.

Problem 22 — List Intersection with Duplicates

Statement: Given two lists of integers, find their intersection including duplicates. Each element in the result should appear as many times as it appears in both lists.

```
import java.util.*;

public class Solution22 {
    public static void main(String[] args) {
        int[] a = {1, 2, 2, 1};
        int[] b = {2, 2};
        System.out.println(intersect(a, b)); // [2, 2]
    }

    static List<Integer> intersect(int[] a, int[] b) {
        Map<Integer, Integer> freqA = new HashMap<>();
        for (int x : a) freqA.put(x, freqA.getOrDefault(x, 0) + 1);

        List<Integer> result = new ArrayList<>();
        for (int x : b) {
            if (freqA.getOrDefault(x, 0) > 0) {
                result.add(x);
                freqA.put(x, freqA.get(x) - 1);
            }
        }

        return result;
    }
}
```

Explanation: A frequency map is built from the first array, then the second array is scanned for matches. Decrementing the frequency on each match ensures duplicates are counted exactly the minimum number of times they appear in both arrays — not over-counted or under-counted.

Problem 23 — Best Time to Buy and Sell Stock

Statement: Given a list of integers representing stock prices on consecutive days, find the maximum profit you can make with at most one buy and one sell. No selling before buying.

```
public class Solution23 {
    public static void main(String[] args) {
        int[] prices = {7, 1, 5, 3, 6, 4};
        System.out.println(maxProfit(prices)); // 5
    }

    static int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
    }
}
```

```

        for (int price : prices) {
            minPrice = Math.min(minPrice, price);
            maxProfit = Math.max(maxProfit, price - minPrice);
        }

        return maxProfit;
    }
}

```

Explanation: This is a single-pass greedy solution, a significant improvement over the $O(n^2)$ brute force of checking every buy-sell pair. `minPrice` tracks the lowest price observed so far (the optimal buy point); `maxProfit` tracks the best achievable profit if selling today.

Since a sale can never precede its purchase, only the minimum price seen up to the current day is relevant — future prices have no bearing on the current decision.

Problem 24 — Minimum Jumps to Reach Last Index

Statement: Given a list of integers, find the minimum number of jumps to reach the last index. Each element represents the maximum jump length from that position.

```

public class Solution24 {
    public static void main(String[] args) {
        int[] arr = {2, 3, 1, 1, 4};
        System.out.println(minJumps(arr));    // 2
    }

    static int minJumps(int[] arr) {
        int jumps = 0, currentEnd = 0, farthest = 0;

        for (int i = 0; i < arr.length - 1; i++) {
            farthest = Math.max(farthest, i + arr[i]);

            if (i == currentEnd) {
                jumps++;
                currentEnd = farthest;
                if (currentEnd >= arr.length - 1) break;
            }
        }

        return jumps;
    }
}

```

Explanation: This is a greedy approach analogous to BFS level traversal. `currentEnd` represents the boundary of the current jump's reach; `farthest` continuously tracks the maximum reach achievable on the next jump as the algorithm scans within the current range.

When the loop index reaches `currentEnd`, a jump is mandatory — `currentEnd` is updated to `farthest`, and the jump counter increments. This mirrors BFS level boundaries: `currentEnd` marks a level's edge, and crossing it initiates a new level (jump).

Java Collections — Solution Reference

Module 2: Map

(HashMap, LinkedHashMap, TreeMap)

Repository: SuperFaculties — DSA Training Materials **Audience:** Faculty Development Program (FDP) — SAGE University **Prepared by:** Nidhish Raj Mourya

Problem 25 — Create and Iterate a HashMap

Statement: Create a HashMap mapping company name to employee count. Add 5 entries. Print each entry using `entrySet()`.

```
import java.util.HashMap;
import java.util.Map;

public class Solution25 {
    public static void main(String[] args) {
        HashMap<String, Integer> company = new HashMap<>();
        company.put("TCS", 5000);
        company.put("Infosys", 3000);
        company.put("Wipro", 4000);
        company.put("HCL", 2000);
        company.put("Accenture", 3500);

        for (Map.Entry<String, Integer> entry : company.entrySet())
        {
            System.out.println(entry.getKey() + ": " +
entry.getValue());
        }
    }
}
```

Explanation: `HashMap<K, V>` stores key-value pairs with unique keys and permits duplicate values. `put(key, value)` overwrites the existing value if the key already exists. `entrySet()` returns a Set of `Map.Entry` objects, exposing both `getKey()` and `getValue()` together.

Important: `HashMap` provides no ordering guarantee. Iteration order depends on internal hash bucket placement and will not necessarily match insertion order.

Problem 26 — Get, containsKey, Remove

Statement: Create a HashMap of student name to marks. Retrieve marks of a given student. Check if "Rohan" exists. Remove "Priya" and print the updated map.

```
import java.util.HashMap;
```

```

public class Solution26 {
    public static void main(String[] args) {
        HashMap<String, Integer> marks = new HashMap<>();
        marks.put("Aman", 85);
        marks.put("Priya", 90);
        marks.put("Rohan", 78);

        System.out.println("Rohan marks: " + marks.get("Rohan"));
        System.out.println("Rohan exists: " +
marks.containsKey("Rohan"));

        marks.remove("Priya");
        System.out.println("Updated: " + marks);
    }
}

```

Explanation: `get(key)` performs an $O(1)$ average-case lookup. If the key is absent, it returns `null` rather than throwing an exception — a notable distinction from `List.get()`, which throws `IndexOutOfBoundsException`.

`containsKey()` is the explicit way to verify key presence, since a `null` return from `get()` is ambiguous: it could indicate either a missing key or a key whose value is genuinely `null`. `remove(key)` deletes the entire key-value pair.

Problem 27 — Character Frequency (Manual)

Statement: Given a `String`, count frequency of each character using a `HashMap`. Print characters and their counts in the order they first appear.

```

import java.util.HashMap;
import java.util.Map;

public class Solution27 {
    public static void main(String[] args) {
        String s = "mississippi";
        HashMap<Character, Integer> freq = new HashMap<>();

        for (char ch : s.toCharArray()) {
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);
        }

        for (Map.Entry<Character, Integer> entry : freq.entrySet())
        {
            System.out.print(entry.getKey() + "=" + entry.getValue()
+ " ");
        }
    }
}

```

Explanation: `s.toCharArray()` converts the `String` to a `char[]`, since `Strings` cannot be iterated character-by-character directly in a `for-each` loop. `Character` (the wrapper class) is required because generic collections cannot hold primitive `char` values — Java performs implicit autoboxing.

Note: Plain HashMap provides no order guarantee. If insertion order must be preserved (as required by the original problem statement), LinkedHashMap should be used instead, as demonstrated in Problem 30.

Problem 28 — Phone Directory

Statement: Build a simple phone directory using HashMap that supports add contact, search by name, and delete contact. Print all contacts sorted by name.

```
import java.util.HashMap;
import java.util.TreeMap;
import java.util.Map;

public class Solution28 {
    public static void main(String[] args) {
        HashMap<String, String> directory = new HashMap<>();

        directory.put("Aman", "9876543210");
        directory.put("Priya", "1234567890");
        directory.put("Raj", "9999999999");

        directory.remove("Priya");

        System.out.println(directory.getOrDefault("Aman", "Not
found"));

        TreeMap<String, String> sorted = new TreeMap<>(directory);
        for (String name : sorted.keySet()) {
            System.out.println(name + ": " + sorted.get(name));
        }
    }
}
```

Explanation: TreeMap is backed by a Red-Black Tree and automatically maintains keys in sorted order. Constructing new TreeMap<>(directory) copies the existing map's entries into a sorted structure.

This is a natural fit for the “sorted by name” requirement: Collections.sort() cannot operate directly on a Map, since Map does not implement the List interface.

Problem 29 — Anagram Check (No Sorting)

Statement: Check if two strings are anagrams using a HashMap. Do not use sorting. Count frequencies of the first string then verify with the second.

```
import java.util.HashMap;

public class Solution29 {
    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent"));    // true
    }

    static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
    }
}
```

```

        HashMap<Character, Integer> freq = new HashMap<>();
        for (char ch : s1.toCharArray()) {
            freq.put(ch, freq.getDefault(ch, 0) + 1);
        }

        for (char ch : s2.toCharArray()) {
            if (!freq.containsKey(ch) || freq.get(ch) == 0) return
false;

            freq.put(ch, freq.get(ch) - 1);
        }

        return true;
    }
}

```

Explanation: This follows the standard frequency-matching pattern. Characters of the first string are counted into a frequency map; characters of the second string then decrement those counts. If any character is missing or its count reaches zero prematurely, the strings cannot be anagrams.

Performance note: The initial length check is an important short-circuit — if lengths differ, an anagram relationship is immediately impossible, avoiding unnecessary computation.

Complexity: $O(n)$ time, $O(1)$ space (bounded by 26 lowercase letters, assuming restricted alphabet input).

Problem 30 — First Non-Repeating Character (LinkedHashMap)

Statement: Given a string, find the first character that appears exactly once. Use LinkedHashMap to preserve insertion order.

```

import java.util.LinkedHashMap;
import java.util.Map;

public class Solution30 {
    public static void main(String[] args) {
        System.out.println(firstNonRepeating("aabbcd")); // c
    }

    static Character firstNonRepeating(String s) {
        LinkedHashMap<Character, Integer> freq = new LinkedHashMap<>
();

        for (char ch : s.toCharArray()) {
            freq.put(ch, freq.getDefault(ch, 0) + 1);
        }

        for (Map.Entry<Character, Integer> entry : freq.entrySet())
        {
            if (entry.getValue() == 1) return entry.getKey();
        }

        return null;
    }
}

```

Explanation: The use of LinkedHashMap is critical here. A plain HashMap's iteration order is unspecified, which would make determining the "first" non-repeating character unreliable. LinkedHashMap internally maintains a doubly-linked list across entries, preserving insertion order without requiring an additional sorting step.

Problem 31 — Word Frequency, Sorted Descending

Statement: Count the frequency of each word in a sentence, case-insensitive. Print in descending order of frequency.

```
import java.util.*;

public class Solution31 {
    public static void main(String[] args) {
        String sentence = "the quick brown fox jumps over the lazy
dog the";

        String[] words = sentence.toLowerCase().split(" ");

        HashMap<String, Integer> freq = new HashMap<>();
        for (String word : words) {
            freq.put(word, freq.getOrDefault(word, 0) + 1);
        }

        List<Map.Entry<String, Integer>> entries = new ArrayList<>
(freq.entrySet());
        entries.sort((a, b) -> b.getValue() - a.getValue());

        for (Map.Entry<String, Integer> entry : entries) {
            System.out.println(entry.getKey() + ": " +
entry.getValue());
        }
    }
}
```

Explanation: `entrySet()` cannot be sorted directly, since it returns a Set (which provides no inherent ordering). The entries must first be copied into an ArrayList, which supports `sort()`.

The lambda `(a, b) -> b.getValue() - a.getValue()` produces descending order. Reversing the subtraction (`a.getValue() - b.getValue()`) would produce ascending order instead.

Problem 32 — Two Sum

Statement: Given an integer array and target T, find two indices whose values sum to T using HashMap. Print the indices.

```
import java.util.HashMap;

public class Solution32 {
    public static void main(String[] args) {
        int[] result = twoSum(new int[]{2, 7, 11, 15}, 9);
        System.out.println(result[0] + ", " + result[1]);    // 0, 1
    }

    static int[] twoSum(int[] nums, int target) {
```



```

        HashMap<Integer, Integer> seen = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (seen.containsKey(complement)) {
                return new int[]{seen.get(complement), i};
            }
            seen.put(nums[i], i);
        }

        return new int[]{-1, -1};
    }
}

```

Explanation: This is one of the most fundamental HashMap patterns in DSA. A brute-force approach checking every pair runs in $O(n^2)$; the HashMap approach reduces this to $O(n)$ in a single pass.

Key insight: the value is used as the map key, and the index as the value — enabling $O(1)$ lookup to check whether the complement of the current element has already been encountered.

Problem 33 — Group Anagrams

Statement: Given a list of strings, group all anagrams together. Each group should be sorted internally. Print all groups.

```

import java.util.*;

public class Solution33 {
    public static void main(String[] args) {
        String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
        System.out.println(groupAnagrams(words));
    }

    static List<List<String>> groupAnagrams(String[] words) {
        HashMap<String, List<String>> groups = new HashMap<>();

        for (String word : words) {
            char[] chars = word.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);

            groups.computeIfAbsent(key, k -> new ArrayList<>
                ()).add(word);
        }

        return new ArrayList<>(groups.values());
    }
}

```

Explanation: Key insight: sorting the characters of anagrams produces an identical string. “eat” → “aet”, “tea” → “aet” — same key, so both belong to the same group.

`computeIfAbsent(key, lambda)` is the idiomatic replacement for the more verbose conditional pattern:

```

if (!groups.containsKey(key)) groups.put(key, new ArrayList<>());
groups.get(key).add(word);

```

If the key is absent, the lambda creates a new ArrayList and registers it in the map; in either case, the returned list is ready for `.add()`.

Problem 34 — Subarray Sum Equals K

Statement: Count the number of subarrays whose sum equals K. Use prefix sum + HashMap for O(n) solution.

```
import java.util.HashMap;

public class Solution34 {
    public static void main(String[] args) {
        int[] nums = {1, 1, 1};
        System.out.println(subarraySum(nums, 2));    // 2
    }

    static int subarraySum(int[] nums, int k) {
        HashMap<Integer, Integer> prefixFreq = new HashMap<>();
        prefixFreq.put(0, 1);

        int count = 0, prefixSum = 0;

        for (int num : nums) {
            prefixSum += num;
            count += prefixFreq.getOrDefault(prefixSum - k, 0);
            prefixFreq.put(prefixSum,
                prefixFreq.getOrDefault(prefixSum, 0) + 1);
        }

        return count;
    }
}
```

Explanation: This is an advanced application combining prefix sums with a HashMap. If $\text{prefixSum}[j] - \text{prefixSum}[i] = k$, then the subarray spanning indices $(i+1, j)$ sums to k .

At each position, the algorithm asks: “how many times have I previously seen a prefix sum of $\text{prefixSum} - k$?” and adds that count to the result.

`prefixFreq.put(0, 1)` is essential — it accounts for the empty prefix ($\text{sum} = 0$) as already seen once, ensuring that subarrays starting from index 0 are correctly counted.

Problem 35 — Longest Consecutive Sequence

Statement: Given an unsorted integer array, find the length of the longest sequence of consecutive integers. Use HashSet for O(n) solution.

```
import java.util.HashSet;
import java.util.Set;

public class Solution35 {
    public static void main(String[] args) {
        int[] nums = {100, 4, 200, 1, 3, 2};
        System.out.println(longestConsecutive(nums));    // 4
    }
}
```

```

    }

    static int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int n : nums) set.add(n);

        int best = 0;
        for (int n : set) {
            if (!set.contains(n - 1)) {
                int curr = n, length = 1;
                while (set.contains(curr + 1)) { curr++; length++; }
                best = Math.max(best, length);
            }
        }
        return best;
    }
}

```

Explanation: This achieves $O(n)$ despite the apparent nested-loop structure. For each number, the algorithm checks whether it is the **start** of a sequence — i.e., $n - 1$ is absent from the set.

If n is a sequence start, the algorithm extends forward ($n+1$, $n+2$, ...) for as long as consecutive values exist, each check being $O(1)$ via the HashSet. Total work remains $O(n)$ overall, since **every number belongs to exactly one sequence**, and the sum of all inner-loop iterations across all outer iterations is bounded by n — the structure is not quadratic despite appearances.

Comparison: A sort-based approach would require $O(n \log n)$; this HashSet-based approach is strictly better.

Problem 36 — LRU Cache (LinkedHashMap)

Statement: Implement an LRU cache of capacity C using LinkedHashMap with access order enabled. Support `get(key)` and `put(key, value)`.

```

import java.util.LinkedHashMap;
import java.util.Map;

public class Solution36 {
    static class LRUCache extends LinkedHashMap<Integer, Integer> {
        int capacity;

        LRUCache(int capacity) {
            super(16, 0.75f, true); // true = access-order mode
            this.capacity = capacity;
        }

        Integer get(int key) {
            return super.getOrDefault(key, -1);
        }

        void put(int key, int value) {
            super.put(key, value);
        }

        @Override

```

```

        protected boolean removeEldestEntry(Map.Entry<Integer,
Integer> eldest) {
            return size() > capacity;
        }
    }

    public static void main(String[] args) {
        LRUCache cache = new LRUCache(2);
        cache.put(1, 1);
        cache.put(2, 2);
        System.out.println(cache.get(1));    // 1
        cache.put(3, 3);                    // evicts key 2
        System.out.println(cache.get(2));    // -1
    }
}

```

Explanation: This is an elegant Java-specific technique. The LinkedHashMap constructor's third parameter, when set to true, enables **access-order** mode — any entry accessed via `get()` or `put()` is internally moved to the end of the linked list, marking it as most recently used.

`removeEldestEntry()` is a protected hook method automatically invoked after every `put()`. Returning true triggers automatic eviction of the least recently used entry.

This implementation derives the entire LRU cache behavior from two built-in LinkedHashMap features — access-order mode and the `removeEldestEntry` hook — without manually implementing a doubly-linked list.

Problem 37 — Top K Frequent Elements

Statement: Given an integer array, return the K most frequent elements. Use bucket sort combined with a frequency HashMap.

```

import java.util.*;

public class Solution37 {
    public static void main(String[] args) {
        int[] nums = {1, 1, 1, 2, 2, 3};
        System.out.println(topK(nums, 2));    // [1, 2]
    }

    static List<Integer> topK(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);

        List<List<Integer>> buckets = new ArrayList<>();
        for (int i = 0; i <= nums.length; i++) buckets.add(new
ArrayList<>());

        for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
            buckets.get(entry.getValue()).add(entry.getKey());
        }

        List<Integer> result = new ArrayList<>();
        for (int i = buckets.size() - 1; i >= 0 && result.size() <
k; i--) {

```

```

        result.addAll(buckets.get(i));
    }

    return result.subList(0, k);
}

```

Explanation: This is a clever application of bucket sort. After computing frequencies via `HashMap`, the algorithm exploits the fact that **maximum frequency cannot exceed `nums.length`**, allowing buckets to be indexed directly by frequency value.

Iterating from the highest-frequency bucket downward and collecting elements until `k` are gathered avoids the $O(n \log n)$ cost of conventional sorting, achieving $O(n)$ overall.

Problem 38 — Isomorphic Strings

Statement: Check if two strings are isomorphic — each character in `s` maps to exactly one character in `t` and vice versa. Use two `HashMaps`.

```

import java.util.HashMap;
import java.util.Map;

public class Solution38 {
    public static void main(String[] args) {
        System.out.println(isIsomorphic("egg", "add")); // true
        System.out.println(isIsomorphic("foo", "bar")); // false
    }

    static boolean isIsomorphic(String s, String t) {
        if (s.length() != t.length()) return false;

        Map<Character, Character> sToT = new HashMap<>();
        Map<Character, Character> tToS = new HashMap<>();

        for (int i = 0; i < s.length(); i++) {
            char cs = s.charAt(i), ct = t.charAt(i);

            if (sToT.containsKey(cs)) {
                if (sToT.get(cs) != ct) return false;
            } else {
                if (tToS.containsKey(ct)) return false;
                sToT.put(cs, ct);
                tToS.put(ct, cs);
            }
        }

        return true;
    }
}

```

Explanation: Isomorphism requires that every character in `s` maps to exactly one character in `t`, **and the reverse mapping also holds** — a true bijection in both directions.

This bidirectional check is the most commonly overlooked requirement. Verifying only the `sToT` mapping is insufficient: if a character in `t` is already mapped from a different character in `s`, the

mapping is invalid even if sToT appears consistent. Both maps are therefore necessary.

For example: "ab" mapped to "aa" — a→a is established, but attempting b→a would conflict since a is already claimed by the mapping from s. The tToS check catches this violation.

Problem 39 — Minimum Window Substring

Statement: Find the minimum length substring of S that contains all characters of T. Use HashMap and sliding window.

```
import java.util.HashMap;
import java.util.Map;

public class Solution39 {
    public static void main(String[] args) {
        System.out.println(minWindow("ADOBECODEBANC", "ABC")); //
BANC
    }

    static String minWindow(String s, String t) {
        Map<Character, Integer> need = new HashMap<>();
        for (char ch : t.toCharArray()) need.put(ch,
need.getDefault(ch, 0) + 1);

        int missing = t.length();
        int left = 0, bestLeft = 0, bestLen = Integer.MAX_VALUE;

        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);
            if (need.containsKey(ch)) {
                need.put(ch, need.get(ch) - 1);
                if (need.get(ch) >= 0) missing--;
            }

            while (missing == 0) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1;
                    bestLeft = left;
                }
                char leftChar = s.charAt(left);
                if (need.containsKey(leftChar)) {
                    need.put(leftChar, need.get(leftChar) + 1);
                    if (need.get(leftChar) > 0) missing++;
                }
                left++;
            }
        }

        return bestLen == Integer.MAX_VALUE ? "" :
s.substring(bestLeft, bestLeft + bestLen);
    }
}
```

Explanation: This is an advanced sliding window problem combined with HashMap-based frequency tracking. The need map stores the required count of each character; missing tracks how many characters are still needed to satisfy the requirement.

The window expands (right advances) until all required characters are present (missing == 0). It then contracts (left advances) while remaining valid, recording the best (smallest) window found at each contraction step.

This **expand-then-shrink** pattern is the canonical approach for variable-size sliding window problems.

Problem 40 — HashMap from Scratch (Separate Chaining)

Statement: Implement a HashMap class from scratch using an array of LinkedLists. Support put, get, and remove. Use default capacity 16 and load factor 0.75.

```
import java.util.LinkedList;

public class Solution40 {
    static class MyHashMap {
        LinkedList<int[]>[] buckets;
        int capacity = 16;

        @SuppressWarnings("unchecked")
        MyHashMap() {
            buckets = new LinkedList[capacity];
            for (int i = 0; i < capacity; i++) buckets[i] = new
LinkedList<>();
        }

        int hash(int key) { return key % capacity; }

        void put(int key, int value) {
            LinkedList<int[]> bucket = buckets[hash(key)];
            for (int[] pair : bucket) {
                if (pair[0] == key) { pair[1] = value; return; }
            }
            bucket.add(new int[]{key, value});
        }

        int get(int key) {
            for (int[] pair : buckets[hash(key)]) {
                if (pair[0] == key) return pair[1];
            }
            return -1;
        }

        void remove(int key) {
            buckets[hash(key)].removeIf(pair -> pair[0] == key);
        }
    }

    public static void main(String[] args) {
        MyHashMap map = new MyHashMap();
        map.put(1, 100);
        map.put(17, 200);    // 1 and 17 collide: 1%16=1, 17%16=1

        System.out.println(map.get(1));    // 100
        System.out.println(map.get(17));   // 200
    }
}
```

```
}
```

Explanation: This implementation illustrates the internal mechanics of HashMap. buckets is an array in which each slot holds a LinkedList — this is the **separate chaining** collision-resolution strategy.

hash(key) = key % capacity is a simplified hash function; the production java.util.HashMap uses more sophisticated bit-spreading to improve key distribution. When keys 1 and 17 collide (both hash to bucket index 1), both entries reside in that bucket's linked list, and lookup requires traversing the chain to find the matching key.

Production context: Since Java 8, when a bucket's chain exceeds 8 entries, it is internally converted to a Red-Black Tree, reducing worst-case lookup from $O(n)$ to $O(\log n)$ per bucket.

Problem 41 — Course Schedule (BFS Topological Sort)

Statement: Given N courses and prerequisite pairs, find the minimum number of semesters needed to complete all or -1 if impossible. Use HashMap and BFS topological sort.

```
import java.util.*;

public class Solution41 {
    public static void main(String[] args) {
        int[][] prereqs = {{1,0},{2,1}};
        System.out.println(minSemesters(3, prereqs)); // 3
    }

    static int minSemesters(int n, int[][] prereqs) {
        Map<Integer, List<Integer>> graph = new HashMap<>();
        int[] indegree = new int[n + 1];

        for (int[] p : prereqs) {
            graph.computeIfAbsent(p[1], k -> new ArrayList<>())
                .add(p[0]);
            indegree[p[0]]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 1; i <= n; i++) {
            if (indegree[i] == 0) queue.offer(i);
        }

        int semesters = 0, completed = 0;

        while (!queue.isEmpty()) {
            semesters++;
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int course = queue.poll();
                completed++;
                for (int next : graph.getOrDefault(course, new ArrayList<>())) {
                    indegree[next]--;
                    if (indegree[next] == 0) queue.offer(next);
                }
            }
        }
    }
}
```



```

    }
}

return completed == n ? semesters : -1;
}
}

```

Explanation: This combines BFS with a Map-based graph representation — known as **Kahn's Algorithm** for topological sorting. graph represents dependencies (which courses become available after completing a given course). indegree[i] tracks how many prerequisites remain for course i; a value of zero means the course is ready to take.

BFS proceeds level-by-level, with each level representing one semester, since all courses with zero remaining prerequisites at that point can be taken in parallel. If completed != n at termination, a cycle exists in the dependency graph, making the schedule impossible — this is the mechanism for detecting circular dependencies.

Problem 42 — Longest Substring Without Repeating Characters

Statement: Given a string, find the length of the longest substring without repeating characters. Use a HashMap to store the last seen index of each character.

```

import java.util.HashMap;
import java.util.Map;

public class Solution42 {
    public static void main(String[] args) {
        System.out.println(lengthOfLongestSubstring("abcabcbb"));
// 3
        System.out.println(lengthOfLongestSubstring("pwwkew"));
// 3
    }

    static int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> lastSeen = new HashMap<>();
        int left = 0, best = 0;

        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);

            if (lastSeen.containsKey(ch) && lastSeen.get(ch) >=
left) {
                left = lastSeen.get(ch) + 1;
            }

            lastSeen.put(ch, right);
            best = Math.max(best, right - left + 1);
        }

        return best;
    }
}

```

Explanation: The lastSeen map stores the most recent index of each character. When a duplicate character is encountered **within the current window**, the window is contracted to begin immediately after the duplicate's prior occurrence.

Critical detail: the condition `lastSeen.get(ch) >= left` is essential — it is not sufficient to know a character was seen before; it must have been seen **within the current window** to count as a true duplicate. If its prior occurrence lies outside the window, it is irrelevant.

Problem 43 — Longest Subarray with Equal 0s and 1s

Statement: Given an array of integers, find the length of the longest subarray with equal number of 0s and 1s. Use prefix sum and HashMap.

```
import java.util.HashMap;
import java.util.Map;

public class Solution43 {
    public static void main(String[] args) {
        int[] arr = {0, 1, 0, 1, 1, 0};
        System.out.println(longestEqual(arr)); // 6
    }

    static int longestEqual(int[] arr) {
        Map<Integer, Integer> prefixIndex = new HashMap<>();
        prefixIndex.put(0, -1);

        int total = 0, best = 0;

        for (int i = 0; i < arr.length; i++) {
            total += (arr[i] == 1) ? 1 : -1;

            if (prefixIndex.containsKey(total)) {
                best = Math.max(best, i - prefixIndex.get(total));
            } else {
                prefixIndex.put(total, i);
            }
        }

        return best;
    }
}
```

Explanation: This is a creative application of the prefix sum technique. **Key insight:** treating each 0 as -1 transforms “equal 0s and 1s” into “subarray summing to 0.”

If two positions show the same running total, the subarray between them sums to zero — indicating an equal-count subarray. `prefixIndex.put(0, -1)` establishes the base case: if a prefix sums to zero from the very start, the entire prefix should be considered valid (treated as if it began at index -1).

Important: only the **first occurrence** of each running total should be stored — overwriting would shorten the maximum subarray length rather than preserving the longest possible span.

Problem 44 — Word Pattern Matching (Bijection)

Statement: Given a list of words and a pattern, find all words that match the pattern. Two words match if there is a bijection between characters of both words.

```
import java.util.HashMap;
import java.util.Map;

public class Solution44 {
    public static void main(String[] args) {
        System.out.println(wordPattern("aab", new String[]{"egg",
"egg", "cheese"})); // true
    }

    static boolean wordPattern(String pattern, String[] words) {
        if (pattern.length() != words.length) return false;

        Map<Character, String> pToW = new HashMap<>();
        Map<String, Character> wToP = new HashMap<>();

        for (int i = 0; i < pattern.length(); i++) {
            char p = pattern.charAt(i);
            String w = words[i];

            if (pToW.containsKey(p)) {
                if (!pToW.get(p).equals(w)) return false;
            } else {
                if (wToP.containsKey(w)) return false;
                pToW.put(p, w);
                wToP.put(w, p);
            }
        }

        return true;
    }
}
```

Explanation: This follows the same bijection logic established in Problem 38 (Isomorphic Strings), applied here to whole words rather than individual characters. Both directional maps (pToW and wToP) are necessary to guarantee a strict one-to-one mapping rather than a many-to-one relationship.

Application context: this pattern is used in log file format validation and data consistency checks, where one needs to verify that one format consistently maps to another.

Problem 45 — Meeting Rooms (Minimum Rooms)

Statement: Given a list of meeting time intervals, find the minimum number of conference rooms required. Use TreeMap to track room availability by time.

```
import java.util.*;

public class Solution45 {
    public static void main(String[] args) {
        int[][] intervals = {{0,30},{5,10},{15,20}};
    }
}
```

```

        System.out.println(minRooms(intervals)); // 2
    }

    static int minRooms(int[][] intervals) {
        if (intervals.length == 0) return 0;

        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        PriorityQueue<Integer> heap = new PriorityQueue<>();

        for (int[] interval : intervals) {
            if (!heap.isEmpty() && heap.peek() <= interval[0]) {
                heap.poll();
            }
            heap.offer(interval[1]);
        }

        return heap.size();
    }
}

```

Explanation: This combines a greedy approach with a min-heap. Sorting intervals by start time enables sequential processing. The min-heap stores the end time of each currently active room.

For each new meeting, the algorithm checks whether the **earliest-freeing room** (the heap's top) has already become available by the new meeting's start time. If so, that room is reused (popped, then a new end time pushed). Otherwise, a new room is required (push only, no pop).

The final **heap size** is the answer — it reflects the maximum number of rooms needed **simultaneously** at any point in the schedule.

Problem 46 — Group Logs by ID, Sort by Timestamp

Statement: Given a list of log entries each as id, timestamp, and message, group logs by id and within each group sort by timestamp. Use HashMap of id to TreeMap.

```

import java.util.*;

public class Solution46 {
    public static void main(String[] args) {
        String[][] logs = {
            {"d1", "9:00", "Get code"},
            {"d2", "9:00", "Start"},
            {"d1", "9:30", "Review code"},
            {"d2", "9:45", "Finish"}
        };

        Map<String, List<String[]>> grouped = new HashMap<>();
        for (String[] log : logs) {
            grouped.computeIfAbsent(log[0], k -> new ArrayList<>
                ()).add(log);
        }

        List<String> sortedIds = new ArrayList<>(grouped.keySet());
        Collections.sort(sortedIds);
    }
}

```

```

        for (String id : sortedIds) {
            List<String[]> group = grouped.get(id);
            group.sort((a, b) -> a[1].compareTo(b[1]));
            for (String[] log : group)
                System.out.println(Arrays.toString(log));
        }
    }
}

```

Explanation: `computeIfAbsent(key, lambda)` is the idiomatic Java pattern for group-by operations. On first encounter of a key, a new `ArrayList` is created and registered; on subsequent encounters, the existing list is returned — in either case, `.add()` can be chained directly.

This demonstrates **two-level sorting**: outer keys (IDs) are sorted first, then each group's entries are independently sorted by timestamp — a nested sorting structure common in grouped-data reporting.

Problem 47 — Single Number (XOR vs HashMap)

Statement: Given an array where every element appears twice except one, find the single element. Solve using XOR first, then verify using `HashMap`.

```

import java.util.HashMap;
import java.util.Map;

public class Solution47 {
    public static void main(String[] args) {
        int[] nums = {4, 1, 2, 1, 2};
        System.out.println("XOR approach: " + singleXOR(nums));
        System.out.println("HashMap approach: " +
singleHashMap(nums));
    }

    static int singleXOR(int[] nums) {
        int result = 0;
        for (int n : nums) result ^= n;
        return result;
    }

    static int singleHashMap(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);

        for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
            if (entry.getValue() == 1) return entry.getKey();
        }
        return -1;
    }
}

```

Explanation: Comparing both approaches is instructive:

- **XOR approach:** $O(n)$ time, **$O(1)$ space**. Exploits the properties $a \oplus a = 0$ and $a \oplus 0 = a$ — XOR-ing all elements cancels out every paired value, leaving only the unpaired one.

- **HashMap approach:** $O(n)$ time, **$O(n)$ space**. More intuitive but requires additional memory.

Interview note: if asked for an $O(1)$ space follow-up, the XOR technique should be presented as it demonstrates a classic application of bit manipulation.

Problem 48 — Design RandomizedSet ($O(1)$ Operations)

Statement: Design a data structure that supports insert, delete, and getRandom in $O(1)$ average time. Use HashMap combined with ArrayList.

```
import java.util.*;

public class Solution48 {
    static class RandomizedSet {
        Map<Integer, Integer> valToIndex = new HashMap<>();
        List<Integer> values = new ArrayList<>();
        Random random = new Random();

        boolean insert(int val) {
            if (valToIndex.containsKey(val)) return false;
            valToIndex.put(val, values.size());
            values.add(val);
            return true;
        }

        boolean remove(int val) {
            if (!valToIndex.containsKey(val)) return false;
            int idx = valToIndex.get(val);
            int lastVal = values.get(values.size() - 1);

            values.set(idx, lastVal);
            valToIndex.put(lastVal, idx);

            values.remove(values.size() - 1);
            valToIndex.remove(val);

            return true;
        }

        int getRandom() {
            return values.get(random.nextInt(values.size()));
        }
    }

    public static void main(String[] args) {
        RandomizedSet rs = new RandomizedSet();
        rs.insert(1); rs.insert(2); rs.insert(3);
        rs.remove(2);
        System.out.println(rs.getRandom());
    }
}
```

Explanation: This is a classic example of the “swap-with-last-then-delete” technique. values (an ArrayList) provides $O(1)$ random access, essential for getRandom(). valToIndex (a HashMap) provides $O(1)$

lookup for both existence checks and position retrieval.

Core technique: removing an arbitrary element from an ArrayList is normally $O(n)$, since subsequent elements must shift. To avoid this, the element to be removed is **swapped with the last element**, then removed from the end — making deletion $O(1)$.

valToIndex must be updated for the element that moved (the former last element) to maintain lookup correctness for future operations.

Problem 49 — Subarray Product Less Than K

Statement: Given a list of integers, find the number of subarrays whose product is less than K. Use a sliding window and track the current product.

```
public class Solution49 {
    public static void main(String[] args) {
        int[] arr = {10, 5, 2, 6};
        System.out.println(numSubarrayProductLessThanK(arr, 100));
    }
}

// 8

static int numSubarrayProductLessThanK(int[] nums, int k) {
    if (k <= 1) return 0;

    int count = 0, product = 1, left = 0;

    for (int right = 0; right < nums.length; right++) {
        product *= nums[right];

        while (product >= k) {
            product /= nums[left];
            left++;
        }

        count += right - left + 1;
    }

    return count;
}
```

Explanation: This is a counting variant of the sliding window pattern. As with sum-based sliding windows, this technique requires all positive values — zero or negative numbers would complicate the monotonic relationship between window size and product.

When $\text{product} \geq k$, the window contracts from the left until the product falls below k again.

Counting technique: for each valid window ending at right , exactly $\text{right} - \text{left} + 1$ valid subarrays exist — since any suffix of the current window (ending at right) also satisfies the product constraint. This counts all valid subarrays in $O(n)$ without explicit enumeration.

Problem 50 — Range Sum Query (Prefix Sum Array)

Statement: Given an array of integers and a list of queries each as a range, find the sum of each range in $O(1)$ after $O(n)$ preprocessing.

```
public class Solution50 {
    static class RangeSum {
        int[] prefix;

        RangeSum(int[] nums) {
            prefix = new int[nums.length + 1];
            for (int i = 0; i < nums.length; i++) {
                prefix[i + 1] = prefix[i] + nums[i];
            }
        }

        int query(int left, int right) {
            return prefix[right + 1] - prefix[left];
        }
    }

    public static void main(String[] args) {
        RangeSum rs = new RangeSum(new int[]{1, 2, 3, 4, 5});
        System.out.println(rs.query(1, 3));    // 9 (2+3+4)
        System.out.println(rs.query(0, 4));    // 15
    }
}
```

Explanation: $prefix[i]$ represents the sum of all elements from index 0 to $i-1$; $prefix[0] = 0$ by convention, representing the empty prefix.

Formula: $sum(left, right) = prefix[right+1] - prefix[left]$ — this reduces any range sum query to $O(1)$ after an initial $O(n)$ preprocessing pass.

Application context: this exact technique underlies cumulative-sum computations in database query optimization (analytics dashboards) and competitive programming problems requiring repeated range queries.

Without prefix sums, each query would cost $O(n)$; with q total queries, that totals $O(n \cdot q)$. Prefix sums reduce this to $O(n + q)$.

End of Module 2 — Map

Java Collections — Solution Reference

Module 3: Queue

(LinkedList, ArrayDeque, PriorityQueue, Deque)

Repository: SuperFaculties — DSA Training Materials **Audience:** Faculty Development Program (FDP) — SAGE University **Prepared by:** Nidhish Raj Mourya

Problem 51 — Basic Queue Operations

Statement: Create a Queue using LinkedList. Enqueue 5 names using offer(). Print size. Dequeue all using poll() until empty. Print each name as it comes out.

```
import java.util.LinkedList;
import java.util.Queue;

public class Solution51 {
    public static void main(String[] args) {
        Queue<String> q = new LinkedList<>();
        q.offer("Aman");
        q.offer("Priya");
        q.offer("Raj");
        q.offer("Neha");
        q.offer("Rohan");

        System.out.println("Size: " + q.size());

        while (!q.isEmpty()) {
            System.out.println(q.poll());
        }
    }
}
```

Explanation: Queue<String> is an interface implemented by LinkedList here. Referencing the collection through its interface type is good practice — switching to ArrayDeque later requires changing only the instantiation line, not the surrounding code.

offer() enqueues to the back. add() performs the same operation but throws an exception when a capacity-bound queue is full; offer() returns false instead, making it the safer choice for bounded queue implementations.

poll() dequeues from the front, returning null if the queue is empty (no exception) — this differs from remove(), which throws NoSuchElementException on an empty queue.

Problem 52 — Peek vs Poll

Statement: Add 5 integers to a Queue. Use peek() 3 times and poll() 3 times alternately. Observe and print how the queue changes.

```
import java.util.LinkedList;
import java.util.Queue;

public class Solution52 {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.offer(10); q.offer(20); q.offer(30); q.offer(40);
        q.offer(50);

        for (int i = 0; i < 3; i++) {
            System.out.println("Peek: " + q.peek());
            System.out.println("Poll: " + q.poll());
            System.out.println("Queue: " + q);
        }
    }
}
```

```
    }  
}
```

Explanation: `peek()` inspects the front element without removing it, returning null on an empty queue. `poll()` both inspects and removes the front element.

Common confusion point: since both operations return the front element, the distinction lies entirely in whether removal occurs — students should be encouraged to trace through state changes explicitly to internalize this difference.

Problem 53 — Reverse a Queue Using a Stack

Statement: Given a Queue of integers, reverse it using only a Stack (ArrayDeque). Print original and reversed queue.

```
import java.util.LinkedList;  
import java.util.Queue;  
import java.util.ArrayDeque;  
import java.util.Deque;  
  
public class Solution53 {  
    public static void main(String[] args) {  
        Queue<Integer> q = new LinkedList<>();  
        for (int i : new int[]{1, 2, 3, 4, 5}) q.offer(i);  
  
        System.out.println("Original: " + q);  
  
        Deque<Integer> stack = new ArrayDeque<>();  
        while (!q.isEmpty()) stack.push(q.poll());  
        while (!stack.isEmpty()) q.offer(stack.pop());  
  
        System.out.println("Reversed: " + q);  
    }  
}
```

Explanation: This is a classic application of opposite-order data structures. Queue is FIFO, Stack is LIFO — the reversal exploits this property directly.

The first loop transfers elements from the queue to the stack, naturally reversing their order (the last queue element becomes the stack's top). The second loop pops from the stack (LIFO — the originally-last element emerges first) and re-enqueues into the queue, completing the reversal.

Problem 54 — Print Queue Without Data Loss

Statement: Print all elements of a Queue without permanently removing them. The queue must be intact after printing.

```
import java.util.LinkedList;  
import java.util.Queue;  
  
public class Solution54 {  
    public static void main(String[] args) {  
        Queue<String> q = new LinkedList<>();
```

```

        q.offer("A"); q.offer("B"); q.offer("C"); q.offer("D");

        for (String item : q) {
            System.out.print(item + " ");
        }

        System.out.println("\nQueue intact: " + q);
    }
}

```

Explanation: LinkedList implements the Iterable interface, allowing a for-each loop to traverse it **without modifying** the underlying queue. This is fundamentally distinct from poll(), which is a destructive read — iteration here is non-destructive.

Problem 55 — Queue Implemented Using Two Stacks

Statement: Implement a Queue using exactly two Stacks. Implement enqueue and dequeue and demonstrate with sample operations.

```

import java.util.ArrayDeque;
import java.util.Deque;

public class Solution55 {
    static class MyQueue {
        Deque<Integer> inStack = new ArrayDeque<>();
        Deque<Integer> outStack = new ArrayDeque<>();

        void enqueue(int x) {
            inStack.push(x);
        }

        Integer dequeue() {
            if (outStack.isEmpty()) {
                while (!inStack.isEmpty()) {
                    outStack.push(inStack.pop());
                }
            }
            return outStack.isEmpty() ? null : outStack.pop();
        }
    }

    public static void main(String[] args) {
        MyQueue q = new MyQueue();
        q.enqueue(1); q.enqueue(2); q.enqueue(3);
        System.out.println(q.dequeue()); // 1
        System.out.println(q.dequeue()); // 2
    }
}

```

Explanation: This implements amortized O(1) dequeue using two stacks. inStack receives newly enqueued elements in LIFO order. When dequeue() is called and outStack is empty, **all** elements are transferred from inStack to outStack — this transfer reverses their order, producing the required FIFO behavior.

If outStack already contains elements, the transfer is skipped entirely. This is the source of the term “amortized” — while a single dequeue operation may occasionally cost O(n), the average cost across many

operations remains $O(1)$.

Problem 56 — Stack Implemented Using Two Queues

Statement: Implement a Stack using two Queues. Implement push and pop and demonstrate with sample operations.

```
import java.util.LinkedList;
import java.util.Queue;

public class Solution56 {
    static class MyStack {
        Queue<Integer> q1 = new LinkedList<>();
        Queue<Integer> q2 = new LinkedList<>();

        void push(int x) {
            q2.offer(x);
            while (!q1.isEmpty()) q2.offer(q1.poll());
            Queue<Integer> temp = q1;
            q1 = q2;
            q2 = temp;
        }

        Integer pop() {
            return q1.isEmpty() ? null : q1.poll();
        }

        Integer top() {
            return q1.isEmpty() ? null : q1.peek();
        }
    }

    public static void main(String[] args) {
        MyStack s = new MyStack();
        s.push(1); s.push(2); s.push(3);
        System.out.println(s.pop());    // 3
        System.out.println(s.top());    // 2
    }
}
```

Explanation: This is the “costly push” approach. Each `push()` places the new element into `q2`, then transfers all existing `q1` elements behind it — positioning the new element at the front. `q1` and `q2` are then swapped, so `q1` always reflects the current stack state with the top element positioned at the front.

`pop()` and `top()` are correspondingly simple, since the queue’s front directly represents the stack’s top.

Problem 57 — Level Order Tree Traversal

Statement: Given a binary tree, print nodes level by level using a Queue. Print each level on a separate line.

```
import java.util.LinkedList;
import java.util.Queue;

public class Solution57 {
```

```

static class Node {
    int val;
    Node left, right;
    Node(int val) { this.val = val; }
}

public static void main(String[] args) {
    Node root = new Node(1);
    root.left = new Node(2);
    root.right = new Node(3);
    root.left.left = new Node(4);
    root.left.right = new Node(5);

    levelOrder(root);
}

static void levelOrder(Node root) {
    if (root == null) return;
    Queue<Node> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int levelSize = q.size();
        StringBuilder level = new StringBuilder();

        for (int i = 0; i < levelSize; i++) {
            Node node = q.poll();
            level.append(node.val).append(" ");
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }

        System.out.println(level.toString().trim());
    }
}

```

Explanation: This is a classic BFS application to tree structures. Capturing `levelSize = q.size()` **before** the inner loop begins is essential — it snapshots how many nodes belong to the current level, before any of their children are enqueued.

The inner loop processes exactly that many nodes, and their children are deferred to the next level's processing. This is precisely the mechanism that enables printing each level independently.

Problem 58 — First Non-Repeating Character in a Stream

Statement: Process a stream of characters. After each character, print the first non-repeating character in the stream so far. Use Queue and HashMap.

```

import java.util.LinkedList;
import java.util.Queue;
import java.util.HashMap;

public class Solution58 {
    public static void main(String[] args) {

```

```

        firstNonRepeatingStream("aabcba");
    }

    static void firstNonRepeatingStream(String stream) {
        Queue<Character> q = new LinkedList<>();
        HashMap<Character, Integer> freq = new HashMap<>();

        for (char ch : stream.toCharArray()) {
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);
            q.offer(ch);

            while (!q.isEmpty() && freq.get(q.peek()) > 1) {
                q.poll();
            }

            System.out.println("After '" + ch + "': " + (q.isEmpty()
? "None" : q.peek()));
        }
    }
}

```

Explanation: This combines Queue and HashMap into a unified pattern. The HashMap tracks character frequencies; the Queue maintains “candidates” — characters that remain unique as far as observed so far.

When a character repeats, it is removed from the front of the queue (if present there), since it no longer qualifies as non-repeating. The front of the queue always represents the current first non-repeating character — or the queue is empty if none currently qualify.

Problem 59 — Circular Queue

Statement: Implement a Circular Queue with fixed size N using an array. Support enqueue, dequeue, isFull, isEmpty with O(1) operations.

```

public class Solution59 {
    static class CircularQueue {
        int[] arr;
        int front, rear, size, capacity;

        CircularQueue(int n) {
            capacity = n;
            arr = new int[n];
            front = rear = -1;
            size = 0;
        }

        boolean enqueue(int val) {
            if (isFull()) return false;
            rear = (rear + 1) % capacity;
            arr[rear] = val;
            if (front == -1) front = 0;
            size++;
            return true;
        }

        Integer dequeue() {

```

```

        if (isEmpty()) return null;
        int val = arr[front];
        front = (front + 1) % capacity;
        size--;
        if (size == 0) front = rear = -1;
        return val;
    }

    boolean isFull() { return size == capacity; }
    boolean isEmpty() { return size == 0; }
}

public static void main(String[] args) {
    CircularQueue cq = new CircularQueue(3);
    cq.enqueue(1); cq.enqueue(2); cq.enqueue(3);
    System.out.println("Full: " + cq.isFull());
    cq.dequeue();
    cq.enqueue(4);
}
}

```

Explanation: The core technique is modulo arithmetic. $(\text{rear} + 1) \% \text{capacity}$ makes the array effectively circular: when rear reaches the array's end, the modulo operation wraps it back to index 0.

This achieves an efficient fixed-size queue without ever shifting elements — purely index arithmetic. The `front == -1` check identifies whether the queue was previously empty, ensuring `front` is set correctly on the first enqueue.

Problem 60 — Sliding Window Maximum (Monotonic Deque)

Statement: Given an array and window size K , find the maximum in each sliding window. Use a Deque for $O(n)$ solution.

```

import java.util.ArrayDeque;
import java.util.Deque;

public class Solution60 {
    public static void main(String[] args) {
        int[] arr = {1, 3, -1, -3, 5, 3, 6, 7};
        int[] result = maxSlidingWindow(arr, 3);
        for (int x : result) System.out.print(x + " ");
    }

    static int[] maxSlidingWindow(int[] nums, int k) {
        Deque<Integer> dq = new ArrayDeque<>();
        int[] result = new int[nums.length - k + 1];

        for (int i = 0; i < nums.length; i++) {
            while (!dq.isEmpty() && dq.peekFirst() < i - k + 1) {
                dq.pollFirst();
            }
            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) {
                dq.pollLast();
            }
            dq.offerLast(i);
        }
    }
}

```

```

        if (i >= k - 1) {
            result[i - k + 1] = nums[dq.peekFirst()];
        }
    }

    return result;
}

```

Explanation: This is one of the most advanced Queue-based patterns, frequently appearing in technical interviews at top-tier product companies. The Deque maintains a **monotonic decreasing** order — the front always holds the current window's maximum.

First while loop: removes indices that have fallen outside the current window's bounds.

Second while loop: removes smaller elements from the back — since a larger incoming element renders previously smaller elements permanently irrelevant (they will never again be the window maximum while the larger element remains in range).

Only indices are stored, not values, to simplify window-boundary comparisons.

Complexity: $O(n)$ overall, since each element is pushed at most once and popped at most once across the entire execution.

Problem 61 — Shortest Path in Grid Avoiding Walls (BFS)

Statement: Given a 2D grid of 0s and 1s where 1 is a wall, find the shortest path from top-left to bottom-right using BFS. Print path length or -1 if unreachable.

```

import java.util.*;

public class Solution61 {
    public static void main(String[] args) {
        int[][] grid = {{0,0,1},{0,0,0},{1,0,0}};
        System.out.println(shortestPath(grid)); // 4
    }

    static int shortestPath(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        if (grid[0][0] == 1 || grid[rows-1][cols-1] == 1) return -1;

        Queue<int[]> queue = new LinkedList<>();
        queue.offer(new int[]{0, 0, 1});
        boolean[][] visited = new boolean[rows][cols];
        visited[0][0] = true;

        int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};

        while (!queue.isEmpty()) {
            int[] curr = queue.poll();
            int r = curr[0], c = curr[1], d = curr[2];

            if (r == rows-1 && c == cols-1) return d;

```



```

        for (int[] dir : dirs) {
            int nr = r + dir[0], nc = c + dir[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols &&
                !visited[nr][nc] && grid[nr][nc] == 0) {
                visited[nr][nc] = true;
                queue.offer(new int[]{nr, nc, d + 1});
            }
        }
    }

    return -1;
}

```

Explanation: BFS guarantees the shortest path in **unweighted graphs or grids**, since it explores level-by-level — the first time a target is reached is guaranteed to be via the shortest possible route.

The `int[] {row, col, distance}` array represents a single search state — a lightweight alternative to defining a custom class in Java for this purpose.

The `visited` array is essential: without it, the same cell could be processed multiple times, leading to incorrect results or excessive computation.

Problem 62 — Rotten Oranges (Multi-Source BFS)

Statement: A grid contains fresh oranges (1), rotten oranges (2), and empty cells (0). Each minute, rotten oranges spread to adjacent fresh ones. Find minimum minutes to rot all oranges or -1.

```

import java.util.*;

public class Solution62 {
    public static void main(String[] args) {
        int[][] grid = {{2,1,1},{1,1,0},{0,1,1}};
        System.out.println( orangesRotting(grid)); // 4
    }

    static int orangesRotting(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        Queue<int[]> queue = new LinkedList<>();
        int fresh = 0;

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] == 2) queue.offer(new int[]{r, c, 0});
                else if (grid[r][c] == 1) fresh++;
            }
        }

        if (fresh == 0) return 0;

        int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};
        int minutes = 0;
    }
}

```

```

        while (!queue.isEmpty()) {
            int[] curr = queue.poll();
            int r = curr[0], c = curr[1], t = curr[2];

            for (int[] dir : dirs) {
                int nr = r + dir[0], nc = c + dir[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols &&
grid[nr][nc] == 1) {
                    grid[nr][nc] = 2;
                    fresh--;
                    minutes = t + 1;
                    queue.offer(new int[]{nr, nc, t + 1});
                }
            }
        }

        return fresh == 0 ? minutes : -1;
    }
}

```

Explanation: This demonstrates the **multi-source BFS** pattern. Unlike standard BFS with one starting point, the queue is initialized with **multiple starting points simultaneously** — every rotten orange — accurately simulating real-world parallel spread.

The grid itself is mutated (grid[nr][nc] = 2) to serve as its own visited marker, eliminating the need for a separate boolean array.

If fresh > 0 remains after BFS completion, some oranges were unreachable — the answer is -1.

Problem 63 — Task Scheduler (Cooldown with PriorityQueue)

Statement: Given tasks as a character array and a cooldown period N, find the minimum number of CPU intervals to finish all tasks. Use PriorityQueue and Queue.

```

import java.util.*;

public class Solution63 {
    public static void main(String[] args) {
        char[] tasks = {'A','A','A','B','B','B'};
        System.out.println(taskScheduler(tasks, 2));    // 8
    }

    static int taskScheduler(char[] tasks, int n) {
        Map<Character, Integer> freq = new HashMap<>();
        for (char t : tasks) freq.put(t, freq.getOrDefault(t, 0) + 1);

        PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
        maxHeap.addAll(freq.values());

        int time = 0;
        Queue<int[]> cooldown = new LinkedList<>();

        while (!maxHeap.isEmpty() || !cooldown.isEmpty()) {

```

```

        time++;

        if (!maxHeap.isEmpty()) {
            int count = maxHeap.poll() - 1;
            if (count > 0) cooldown.offer(new int[]{time + n,
count});
        }

        if (!cooldown.isEmpty() && cooldown.peek()[0] == time) {
            maxHeap.offer(cooldown.poll()[1]);
        }
    }

    return time;
}
}

```

Explanation: This combines a PriorityQueue and a Queue, each serving a distinct purpose. The maxHeap always returns the task with the highest remaining frequency — the greedy rationale is to schedule the most frequent task first, maximizing benefit from its cooldown window.

The cooldown queue tracks when each task becomes available again (time + n). At each time unit, one task executes (from the heap), and the algorithm checks whether any cooled-down task is now available to rejoin the heap.

This is a simplified model of CPU scheduling algorithms used in real operating systems, which similarly combine priority and waiting-queue concepts.

Problem 64 — Reconstruct Queue by Height

Statement: Given people described as height and k where k is the number of taller or equal people in front, reconstruct the original queue using PriorityQueue.

```

import java.util.*;

public class Solution64 {
    public static void main(String[] args) {
        int[][] people = {{7,0},{4,4},{7,1},{5,0},{6,1},{5,2}};
        int[][] result = reconstructQueue(people);
        for (int[] p : result)
            System.out.println(Arrays.toString(p));
    }

    static int[][] reconstructQueue(int[][] people) {
        Arrays.sort(people, (a, b) -> a[0] != b[0] ? b[0] - a[0] :
a[1] - b[1]);

        List<int[]> result = new ArrayList<>();
        for (int[] person : people) {
            result.add(person[1], person);
        }

        return result.toArray(new int[0][]);
    }
}

```

```
}
```

Explanation: This is a clever application of greedy insertion. **Sorting strategy:** process the tallest people first, since taller individuals are unaffected by the presence of shorter people regarding their “k” count.

Starting from the tallest and inserting progressively, each new insertion only affects the “k” requirements of individuals already present in the list — who are guaranteed to be at least as tall. Future, shorter insertions cannot affect those existing requirements.

`result.add(person[1], person)` inserts at exactly the k-th index, ensuring precisely k taller-or-equal individuals precede it.

Problem 65 — Kth Largest Element in a Stream

Statement: Design a class that returns the Kth largest element in a stream after each new value is added. Use a min-heap of size K.

```
import java.util.PriorityQueue;

public class Solution65 {
    static class KthLargest {
        PriorityQueue<Integer> minHeap;
        int k;

        KthLargest(int k) {
            this.k = k;
            minHeap = new PriorityQueue<>();
        }

        Integer add(int val) {
            minHeap.offer(val);
            if (minHeap.size() > k) minHeap.poll();
            return minHeap.size() == k ? minHeap.peek() : null;
        }
    }

    public static void main(String[] args) {
        KthLargest kl = new KthLargest(3);
        int[] stream = {4, 5, 8, 2, 7};
        for (int v : stream) System.out.println(kl.add(v));
    }
}
```

Explanation: A min-heap of size k always retains the **top k largest elements observed so far**, and the heap’s top (`peek()`) — being the smallest among those k — is exactly the Kth largest.

On each addition, the new value is pushed; if the heap size exceeds k, the smallest is removed, maintaining exactly k elements at all times.

This streaming pattern is broadly applicable wherever a continuously updated answer is required without reprocessing the entire dataset on each query.

Problem 66 — Merge K Sorted Lists Using PriorityQueue

Statement: Given K sorted arrays, merge them into a single sorted array using PriorityQueue. Achieve $O(N \log K)$ complexity.

```
import java.util.*;

public class Solution66 {
    public static void main(String[] args) {
        int[][] lists = {{1,4,7},{2,5,8},{3,6,9}};
        System.out.println(mergeKSorted(lists));
    }

    static List<Integer> mergeKSorted(int[][] lists) {
        PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) ->
a[0] - b[0]);

        for (int i = 0; i < lists.length; i++) {
            if (lists[i].length > 0) heap.offer(new int[]{lists[i]
[0], i, 0});
        }

        List<Integer> result = new ArrayList<>();

        while (!heap.isEmpty()) {
            int[] curr = heap.poll();
            result.add(curr[0]);

            int listIdx = curr[1], elemIdx = curr[2];
            if (elemIdx + 1 < lists[listIdx].length) {
                heap.offer(new int[]{lists[listIdx][elemIdx + 1],
listIdx, elemIdx + 1});
            }
        }

        return result;
    }
}
```

Explanation: This achieves $O(N \log K)$, a substantial improvement over the $O(NK)$ brute-force alternative. The heap holds, at any time, only the **current smallest unprocessed element from each list** — at most K elements, not N.

When the smallest element is popped (the current minimum across all lists), that list's **next element** is pushed onto the heap.

Since the heap never holds more than K elements at once, each operation costs $O(\log K)$, and N total elements are processed — giving combined $O(N \log K)$, which remains efficient even as K grows.

Problem 67 — Implement Deque from Scratch (Doubly Linked List)

Statement: Implement a Deque using a doubly linked list with $O(1)$ insertFront, insertRear, deleteFront, deleteRear, peekFront, peekRear operations.

```

public class Solution67 {
    static class DNode {
        int val;
        DNode prev, next;
        DNode(int val) { this.val = val; }
    }

    static class MyDeque {
        DNode head, tail;
        int size = 0;

        void addFirst(int val) {
            DNode node = new DNode(val);
            if (head == null) { head = tail = node; }
            else { node.next = head; head.prev = node; head = node;

            size++;
        }

        void addLast(int val) {
            DNode node = new DNode(val);
            if (tail == null) { head = tail = node; }
            else { tail.next = node; node.prev = tail; tail = node;

            size++;
        }

        Integer removeFirst() {
            if (head == null) return null;
            int val = head.val;
            head = head.next;
            if (head != null) head.prev = null;
            else tail = null;
            size--;
            return val;
        }

        Integer removeLast() {
            if (tail == null) return null;
            int val = tail.val;
            tail = tail.prev;
            if (tail != null) tail.next = null;
            else head = null;
            size--;
            return val;
        }
    }

    public static void main(String[] args) {
        MyDeque dq = new MyDeque();
        dq.addLast(1); dq.addLast(2); dq.addFirst(0);
        System.out.println(dq.removeFirst()); // 0
        System.out.println(dq.removeLast()); // 2
    }
}

```

Explanation: This demonstrates the internal mechanism enabling Deque's $O(1)$ operations at both ends. A doubly linked list — each node maintaining both prev and next pointers — permits $O(1)$ insertion and deletion from **either direction**.

A singly linked list would require $O(n)$ for `addLast()` (requiring full traversal to reach the tail), but maintaining an explicit tail pointer makes this $O(1)$.

Edge cases requiring careful handling: when the list becomes empty (head or tail becomes null), both pointers must be updated consistently.

Problem 68 — Sliding Window Maximum: Brute Force vs Optimal

Statement: Given an array and window size K , print the maximum of every contiguous subarray of size K using a monotonic Deque.

```
import java.util.*;

public class Solution68 {
    public static void main(String[] args) {
        int[] arr = {2, 3, 1, 1, 5};
        System.out.println("Brute: " +
Arrays.toString(slidingMaxBrute(arr, 3)));
        System.out.println("Optimal: " +
Arrays.toString(slidingMaxOptimal(arr, 3)));
    }

    static int[] slidingMaxBrute(int[] arr, int k) {
        int[] result = new int[arr.length - k + 1];
        for (int i = 0; i <= arr.length - k; i++) {
            int max = arr[i];
            for (int j = i; j < i + k; j++) max = Math.max(max,
arr[j]);
            result[i] = max;
        }
        return result;
    }

    static int[] slidingMaxOptimal(int[] arr, int k) {
        Deque<Integer> dq = new ArrayDeque<>();
        int[] result = new int[arr.length - k + 1];

        for (int i = 0; i < arr.length; i++) {
            while (!dq.isEmpty() && dq.peekFirst() < i - k + 1)
dq.pollFirst();
            while (!dq.isEmpty() && arr[dq.peekLast()] < arr[i])
dq.pollLast();
            dq.offerLast(i);
            if (i >= k - 1) result[i - k + 1] = arr[dq.peekFirst()];
        }

        return result;
    }
}
```

Explanation: This side-by-side comparison is well-suited for classroom demonstration of algorithmic complexity differences.

Brute force: an inner loop scans all k elements for every window — $O(n \times k)$. For large k , this becomes prohibitively slow.

Optimal: a monotonic deque maintains the current window's maximum at the front at all times — $O(n)$, since each element is pushed once and popped at most once total.

Demonstration suggestion for faculty: with $k=1000$ and $n=100,000$, brute force performs approximately 10^8 operations, while the deque approach performs approximately 10^5 — illustrating the practical impact of $O(n \times k)$ versus $O(n)$.

Problem 69 — Connect Nodes at Same Level (Next Right Pointer)

Statement: Given a binary tree, connect all nodes at the same level using BFS. Each node should point to its next right neighbor. NULL if it is the last node in its level.

```
import java.util.*;

public class Solution69 {
    static class Node {
        int val;
        Node left, right, next;
        Node(int val) { this.val = val; }
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);
        root.left.left = new Node(4);
        root.left.right = new Node(5);
        root.right.left = new Node(6);
        root.right.right = new Node(7);

        connect(root);
        System.out.println(root.left.next.val);    // 3
    }

    static Node connect(Node root) {
        if (root == null) return root;

        Queue<Node> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            Node prev = null;

            for (int i = 0; i < size; i++) {
                Node node = queue.poll();
                if (prev != null) prev.next = node;
                prev = node;

                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }

        return root;
    }
}
```



```
    }
}
```

Explanation: This extends the level-order traversal pattern from Problem 57 — structurally identical, but each level's nodes are additionally linked to one another via the next pointer.

prev tracks the previous node within the current level; as each new node is dequeued, it is linked from the previous one.

Application context: this mirrors how tree visualization tools implement sibling navigation, or how organizational chart software links same-level employees.

Problem 70 — Shortest Path in Maze (BFS, Four Directions)

Statement: Given a maze as a 2D grid, find the shortest path from source to destination avoiding obstacles. Each step can move in four directions. Use BFS.

```
import java.util.*;

public class Solution70 {
    public static void main(String[] args) {
        int[][] maze = {{0,0,1,0},{0,0,0,0},{1,0,1,0},{0,0,0,0}};
        System.out.println(mazePath(maze, new int[]{0,0}, new int[]
{3,3})); // 6
    }

    static int mazePath(int[][] maze, int[] src, int[] dest) {
        int rows = maze.length, cols = maze[0].length;
        Queue<int[]> queue = new LinkedList<>();
        queue.offer(new int[]{src[0], src[1], 0});

        boolean[][] visited = new boolean[rows][cols];
        visited[src[0]][src[1]] = true;

        int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};

        while (!queue.isEmpty()) {
            int[] curr = queue.poll();
            int r = curr[0], c = curr[1], dist = curr[2];

            if (r == dest[0] && c == dest[1]) return dist;

            for (int[] dir : dirs) {
                int nr = r + dir[0], nc = c + dir[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols &&
                    maze[nr][nc] == 0 && !visited[nr][nc]) {
                    visited[nr][nc] = true;
                    queue.offer(new int[]{nr, nc, dist + 1});
                }
            }
        }

        return -1;
    }
}
```

Explanation: This is structurally similar to Problem 61's BFS approach, generalized to allow source and destination at any grid position rather than fixed corners.

The four directions explicitly enumerated — up, down, left, right — would expand to eight if diagonal movement were permitted.

BFS guarantees the shortest path for **unweighted edges**, where each step is treated as having equal cost (1).

Problem 71 — Running Median (Two Heaps)

Statement: Given a stream of integers, design a data structure that returns the median after each insertion. Use two PriorityQueues — one max-heap and one min-heap.

```
import java.util.*;

public class Solution71 {
    static class MedianFinder {
        PriorityQueue<Integer> lo = new PriorityQueue<>
(Collections.reverseOrder());
        PriorityQueue<Integer> hi = new PriorityQueue<>();

        void addNum(int num) {
            lo.offer(num);
            hi.offer(lo.poll());

            if (hi.size() > lo.size()) {
                lo.offer(hi.poll());
            }
        }

        double findMedian() {
            if (lo.size() > hi.size()) return lo.peek();
            return (lo.peek() + hi.peek()) / 2.0;
        }
    }

    public static void main(String[] args) {
        MedianFinder mf = new MedianFinder();
        int[] stream = {5, 15, 1, 3};
        for (int n : stream) {
            mf.addNum(n);
            System.out.println("After adding " + n + ": median = " +
mf.findMedian());
        }
    }
}
```

Explanation: This is the dual-heap pattern, an advanced-level technique. lo (a max-heap) holds the lower half of observed numbers; hi (a min-heap) holds the upper half.

Balance invariant: lo's size is always equal to or exactly one greater than hi's size, simplifying the median computation.

Insertion procedure: the new number is first pushed to lo; its maximum is then moved to hi (ensuring lo's maximum never exceeds hi's minimum); size balance is then re-verified.

Median computation: if l_o is larger, its top is the median (odd total count); if sizes are equal, the average of both tops is the median (even total count).

Problem 72 — Job Scheduling for Maximum Profit

Statement: Given a list of N jobs each with a deadline and profit, find the maximum profit by selecting at most one job per time slot. Use a PriorityQueue sorted by profit descending.

```
import java.util.*;

public class Solution72 {
    public static void main(String[] args) {
        int[][] jobs = {{2,100},{1,50},{2,10},{1,20},{3,30}};
        System.out.println(maxProfitJobs(jobs)); // 180
    }

    static int maxProfitJobs(int[][] jobs) {
        Arrays.sort(jobs, (a, b) -> a[0] - b[0]);

        int maxDeadline = 0;
        for (int[] job : jobs) maxDeadline = Math.max(maxDeadline,
job[0]);

        PriorityQueue<Integer> heap = new PriorityQueue<>();
        int j = jobs.length - 1, total = 0;

        for (int slot = maxDeadline; slot >= 1; slot--) {
            while (j >= 0 && jobs[j][0] >= slot) {
                heap.offer(jobs[j][1]);
                j--;
            }
            if (!heap.isEmpty()) total += heap.poll();
        }

        return total;
    }
}
```

Explanation: This combines a greedy approach with a heap in a non-obvious arrangement. Jobs are first sorted by deadline; time slots are then processed **in reverse, from latest to earliest**.

For each slot, all jobs whose deadline is at or beyond that slot are added to the eligible pool (heap). The heap then yields the highest-profit eligible job for that slot — assigning it and leaving lower-profit jobs available for earlier slots in subsequent iterations.

Problem 73 — Snake and Ladder (Minimum Dice Rolls via BFS)

Statement: Implement a snake and ladder game simulation using BFS. Given board positions for snakes and ladders, find the minimum number of dice rolls to reach position 100.

```
import java.util.*;
```

```

public class Solution73 {
    public static void main(String[] args) {
        Map<Integer, Integer> board = new HashMap<>();
        board.put(3, 22); board.put(5, 8); board.put(11, 26);
        board.put(20, 29); board.put(17, 4); board.put(19, 7);

        System.out.println(snakesLadders(board));
    }

    static int snakesLadders(Map<Integer, Integer> board) {
        Queue<int[]> queue = new LinkedList<>();
        queue.offer(new int[]{1, 0});
        Set<Integer> visited = new HashSet<>();
        visited.add(1);

        while (!queue.isEmpty()) {
            int[] curr = queue.poll();
            int pos = curr[0], rolls = curr[1];

            for (int dice = 1; dice <= 6; dice++) {
                int next = pos + dice;
                if (next > 100) break;

                next = board.getOrDefault(next, next);

                if (next == 100) return rolls + 1;

                if (!visited.contains(next)) {
                    visited.add(next);
                    queue.offer(new int[]{next, rolls + 1});
                }
            }
        }

        return -1;
    }
}

```

Explanation: This demonstrates a practical application of BFS to a board-game scenario. The board is modeled as a **graph**, where each position is a node and dice roll outcomes (1 through 6) form the edges.

BFS guarantees a minimum-rolls answer, since it explores level-by-level, with each level corresponding to one additional dice roll.

`board.getOrDefault(next, next)` is an elegant idiom: it returns the snake/ladder destination if one exists at that position, or the position itself otherwise (a no-op fallback).

Problem 74 — Word Ladder: All Shortest Transformation Sequences

Statement: Given a word ladder dictionary, find all shortest transformation sequences from `beginWord` to `endWord`. Use BFS level by level with a deque of paths.

```

import java.util.*;

```

```

    public class Solution74 {
        public static void main(String[] args) {
            List<String> wordList =
Arrays.asList("hot","dot","dog","lot","log","cog");
            System.out.println(findLadders("hit", "cog", wordList));
        }

        static List<List<String>> findLadders(String begin, String end,
List<String> wordList) {
            Set<String> wordSet = new HashSet<>(wordList);
            if (!wordSet.contains(end)) return new ArrayList<>();

            Map<String, List<String>> parents = new HashMap<>();
            Set<String> currentLevel = new HashSet<>();
            currentLevel.add(begin);
            boolean found = false;

            while (!currentLevel.isEmpty() && !found) {
                wordSet.removeAll(currentLevel);
                Set<String> nextLevel = new HashSet<>();

                for (String word : currentLevel) {
                    char[] chars = word.toCharArray();
                    for (int i = 0; i < chars.length; i++) {
                        char original = chars[i];
                        for (char c = 'a'; c <= 'z'; c++) {
                            chars[i] = c;
                            String newWord = new String(chars);
                            if (wordSet.contains(newWord)) {
                                nextLevel.add(newWord);
                                parents.computeIfAbsent(newWord, k ->
new ArrayList<>()).add(word);
                                if (newWord.equals(end)) found = true;
                            }
                        }
                        chars[i] = original;
                    }
                }
                currentLevel = nextLevel;
            }

            List<List<String>> result = new ArrayList<>();
            if (found) {
                backtrack(end, begin, parents, new LinkedList<>(),
result);
            }
            return result;
        }

        static void backtrack(String word, String begin, Map<String,
List<String>> parents,
                                LinkedList<String> path,
List<List<String>> result) {
            path.addFirst(word);
            if (word.equals(begin)) {
                result.add(new ArrayList<>(path));
            } else {
                for (String parent : parents.getOrDefault(word, new
ArrayList<>())) {
                    backtrack(parent, begin, parents, path, result);
                }
            }
        }
    }

```

```

        }
    }
    path.removeFirst();
}
}

```

Explanation: This is the most advanced Queue/BFS problem in this module. Rather than finding only the shortest path length, this problem requires finding **all** shortest paths simultaneously — necessitating a modified BFS structure.

The parents map tracks, for each word, which words led to it (multiple parents may exist if multiple equal-length paths converge on the same word). BFS proceeds level-by-level (`currentLevel` → `nextLevel`); as soon as end is discovered within a level, the search halts — BFS's level-order property guarantees this is the first (shortest) level at which end could be found.

Backtracking then reconstructs all possible paths from end back to begin using the parents map.

End of Module 3 — Queue

Java Collections — Solution Reference

Module 4: Set

(HashSet, LinkedHashSet, TreeSet)

Repository: SuperFaculties — DSA Training Materials **Audience:** Faculty Development Program (FDP) — SAGE University **Prepared by:** Nidhish Raj Mourya

Problem 75 — Create a HashSet (No Duplicates)

Statement: Create a HashSet of integers. Try adding [1, 2, 3, 2, 4, 1, 5]. Print the set. Observe that duplicates are not stored.

```

import java.util.HashSet;

public class Solution75 {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();
        int[] input = {1, 2, 3, 2, 4, 1, 5};

        for (int x : input) set.add(x);

        System.out.println(set);
        System.out.println("Size: " + set.size());
    }
}

```

Explanation: The Set interface guarantees **no duplicates**. Calling `add()` with an already-present value is silently ignored — no exception is thrown; the method simply returns `false` (not used here, but available for explicit duplicate detection).

`HashSet` provides no ordering guarantee — internally, it is backed by a `HashMap` (using a dummy placeholder value for each key).

Problem 76 — Contains and Remove

Statement: Create a `HashSet` of 5 city names. Check if “Indore” exists using `contains()`. Remove “Delhi”. Print the final size.

```
import java.util.HashSet;

public class Solution76 {
    public static void main(String[] args) {
        HashSet<String> cities = new HashSet<>();
        cities.add("Delhi"); cities.add("Mumbai");
        cities.add("Indore");
        cities.add("Pune"); cities.add("Jaipur");

        System.out.println("Indore exists: " +
            cities.contains("Indore"));
        cities.remove("Delhi");
        System.out.println("Final size: " + cities.size());
    }
}
```

Explanation: `contains()` runs in $O(1)$ average case, leveraging the same hashing mechanism as `HashMap` key lookups. `remove(value)` silently does nothing if the value is absent — no exception is raised.

Problem 77 — HashSet vs LinkedHashSet vs TreeSet

Statement: Add the same 5 elements to a `HashSet`, a `LinkedHashSet`, and a `TreeSet`. Print elements of each. Observe insertion order vs natural order.

```
import java.util.*;

public class Solution77 {
    public static void main(String[] args) {
        int[] nums = {5, 2, 8, 1, 9, 3};

        Set<Integer> hashSet = new HashSet<>();
        Set<Integer> linkedHashSet = new LinkedHashSet<>();
        Set<Integer> treeSet = new TreeSet<>();

        for (int n : nums) {
            hashSet.add(n);
            linkedHashSet.add(n);
            treeSet.add(n);
        }

        System.out.println("HashSet: " + hashSet);
        System.out.println("LinkedHashSet: " + linkedHashSet);
        System.out.println("TreeSet: " + treeSet);
    }
}
```

```
    }
}
```

Explanation: This is one of the most important comparisons for interview preparation:

Implementation	Internal Structure	Ordering	Time Complexity
HashSet	HashMap	None	O(1) average
LinkedHashSet	HashMap + linked list	Insertion order	O(1) average
TreeSet	Red-Black Tree	Sorted (natural / Comparator)	O(log n)

Guidance for instruction: “When order does not matter and speed is the priority, use HashSet. When insertion order must be preserved, use LinkedHashSet. When automatically sorted output is required, use TreeSet.”

Problem 78 — Deduplicate List, Preserve Order

Statement: Given an ArrayList with duplicates, deduplicate it using a Set and convert back to a list. Preserve insertion order.

```
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;

public class Solution78 {
    public static void main(String[] args) {
        List<String> lst = new ArrayList<>(List.of("a", "b", "a",
"c", "b", "d"));

        LinkedHashSet<String> deduped = new LinkedHashSet<>(lst);
        List<String> result = new ArrayList<>(deduped);

        System.out.println(result);    // [a, b, c, d]
    }
}
```

Explanation: The LinkedHashSet constructor directly accepts a Collection, allowing deduplication with order preservation in a single line. This is the Set-section equivalent of the technique seen earlier in the List module (Problem 9), here demonstrating an explicit constructor-based pattern.

Problem 79 — Union (Manual, Without addAll)

Statement: Given two HashSets of integers, find their union without using addAll(). Manually iterate both sets and build the result set.

```
import java.util.HashSet;
import java.util.Set;
```



```

public class Solution79 {
    public static void main(String[] args) {
        Set<Integer> a = new HashSet<>(Set.of(1, 2, 3, 4));
        Set<Integer> b = new HashSet<>(Set.of(3, 4, 5, 6));

        Set<Integer> result = new HashSet<>();
        for (int x : a) result.add(x);
        for (int x : b) result.add(x);

        System.out.println("Union: " + result);
    }
}

```

Explanation: This manual implementation illustrates what `addAll()` performs internally — both sets' elements are added to a new `Set`, with the `Set`'s inherent uniqueness property automatically eliminating duplicates.

Built-in equivalent: `result.addAll(a); result.addAll(b);` accomplishes the same outcome more concisely.

Problem 80 — Intersection (Manual)

Statement: Find intersection of two sets using `retainAll()` approach AND a manual approach (iterate + contains). Compare outputs.

```

import java.util.HashSet;
import java.util.Set;

public class Solution80 {
    public static void main(String[] args) {
        Set<Integer> a = new HashSet<>(Set.of(1, 2, 3, 4));
        Set<Integer> b = new HashSet<>(Set.of(3, 4, 5, 6));

        Set<Integer> result = new HashSet<>();
        for (int x : a) {
            if (b.contains(x)) result.add(x);
        }

        System.out.println("Intersection: " + result);    // {3, 4}
    }
}

```

Explanation: Iterating one set while checking membership (`contains()`) in the other identifies the intersection. The built-in alternative, `a.retainAll(b)`, accomplishes this directly — but **modifies a in place**. If the original contents of a must be preserved, a defensive copy should be created beforehand.

Problem 81 — Difference (Manual, then Verified)

Statement: Find A-B (elements in A but not in B) without using `removeAll()` — do it manually. Then verify with `removeAll()`.

```

import java.util.HashSet;
import java.util.Set;

public class Solution81 {

```

```

public static void main(String[] args) {
    Set<Integer> a = new HashSet<>(Set.of(1, 2, 3, 4, 5));
    Set<Integer> b = new HashSet<>(Set.of(3, 4));

    Set<Integer> manual = new HashSet<>();
    for (int x : a) {
        if (!b.contains(x)) manual.add(x);
    }
    System.out.println("Manual A-B: " + manual);

    Set<Integer> usingRemoveAll = new HashSet<>(a);
    usingRemoveAll.removeAll(b);
    System.out.println("removeAll A-B: " + usingRemoveAll);
}
}

```

Explanation: $A - B$ (elements in A absent from B) is computed manually for instructional clarity, then verified against the built-in `removeAll()`. **Caution:** `removeAll()` modifies its receiver directly — a defensive copy (`new HashSet<>(a)`) is necessary if the original set must remain unmodified.

Problem 82 — Subset Check

Statement: Check if Set A is a subset of Set B. Implement manually (iterate every element of A and check in B). Then use `containsAll()`.

```

import java.util.HashSet;
import java.util.Set;

public class Solution82 {
    public static void main(String[] args) {
        Set<Integer> a = new HashSet<>(Set.of(2, 4));
        Set<Integer> b = new HashSet<>(Set.of(1, 2, 3, 4, 5));

        boolean manual = true;
        for (int x : a) {
            if (!b.contains(x)) { manual = false; break; }
        }
        System.out.println("Manual subset: " + manual);

        System.out.println("containsAll: " + b.containsAll(a));
    }
}

```

Explanation: A subset relationship requires every element of A to be present in B. `b.containsAll(a)` is the built-in equivalent — internally, it performs the same per-element `contains()` check in a loop, but offers cleaner, more readable syntax.

Problem 83 — Distinct Words, Sorted

Statement: Given a paragraph, find all distinct words (case-insensitive). Use `TreeSet` to return them in alphabetical order.

```

import java.util.HashSet;
import java.util.Set;
import java.util.TreeSet;

```

```

import java.util.Arrays;

public class Solution83 {
    public static void main(String[] args) {
        String para = "the quick brown fox jumps over the lazy dog";

        Set<String> words = new HashSet<>
(Arrays.asList(para.toLowerCase().split(" ")));
        Set<String> sorted = new TreeSet<>(words);

        System.out.println(sorted);
    }
}

```

Explanation: `para.toLowerCase().split(" ")` normalizes case before splitting on whitespace, yielding a `String[]`. `Arrays.asList()` provides a List view of the array suitable for the `HashSet` constructor.

When a `TreeSet` constructor receives an existing collection, it automatically sorts the elements using its internal Red-Black Tree structure during insertion.

Problem 84 — Longest Consecutive Sequence

Statement: Given an integer array, find the length of the longest sequence of consecutive integers using `HashSet`. For each sequence start, extend the streak forward.

```

import java.util.HashSet;
import java.util.Set;

public class Solution84 {
    public static void main(String[] args) {
        int[] nums = {100, 4, 200, 1, 3, 2};
        System.out.println(longestConsecutive(nums)); // 4
    }

    static int longestConsecutive(int[] nums) {
        Set<Integer> numSet = new HashSet<>();
        for (int n : nums) numSet.add(n);

        int best = 0;

        for (int n : numSet) {
            if (!numSet.contains(n - 1)) {
                int curr = n;
                int length = 1;
                while (numSet.contains(curr + 1)) {
                    curr++;
                    length++;
                }
                best = Math.max(best, length);
            }
        }

        return best;
    }
}

```

Explanation: This achieves $O(n)$ despite an apparent nested-loop structure. **Key insight:** for each number, check whether it represents the **start** of a sequence — that is, $n - 1$ is absent from the set.

If n is a sequence start, the algorithm extends forward ($n+1$, $n+2$, ...) as far as consecutive values exist, with each check $O(1)$ via the HashSet. Total operations remain $O(n)$ because **each number belongs to exactly one sequence**, and the cumulative inner-loop work across all outer iterations is bounded by n overall — this is not a quadratic algorithm despite its surface appearance.

Comparison: sorting first would cost $O(n \log n)$; this HashSet-based approach is strictly better.

Problem 85 — All Pairs with a Given Difference

Statement: Given a list of integers, find all pairs whose difference is exactly K . Use a HashSet for $O(n)$ solution. Print each pair only once.

```
import java.util.*;

public class Solution85 {
    public static void main(String[] args) {
        int[] arr = {1, 5, 3, 4, 2};
        System.out.println(pairsWithDiffK(arr, 2));
    }

    static List<int[]> pairsWithDiffK(int[] arr, int k) {
        Set<Integer> seen = new HashSet<>();
        for (int x : arr) seen.add(x);

        Set<String> added = new HashSet<>();
        List<int[]> result = new ArrayList<>();

        for (int x : arr) {
            if (seen.contains(x + k)) {
                int small = Math.min(x, x + k), large = Math.max(x,
x + k);

                String key = small + "," + large;
                if (!added.contains(key)) {
                    result.add(new int[]{small, large});
                    added.add(key);
                }
            }
        }

        result.sort((a, b) -> a[0] - b[0]);
        return result;
    }
}
```

Explanation: The seen set provides $O(1)$ membership checking — instantly answering “does $x + k$ exist in the array?”

Avoiding duplicate pairs requires care. A pair could be discovered from either direction (e.g., when processing the smaller element, or when processing the larger one). Encoding each pair as a normalized “min,max” string key ensures both directions map to the same identifier, preventing duplicate entries.

Problem 86 — Custom Comparable for TreeSet Ordering

Statement: Create a TreeSet of Student objects sorted by CGPA descending. Implement Comparable in the Student class. Add 5 students and print in CGPA order.

```
import java.util.*;

public class Solution86 {
    static class Student implements Comparable<Student> {
        String name;
        double cgpa;

        Student(String name, double cgpa) { this.name = name;
this.cgpa = cgpa; }

        @Override
        public int compareTo(Student other) {
            if (this.cgpa != other.cgpa) return
Double.compare(other.cgpa, this.cgpa);
            return this.name.compareTo(other.name);
        }

        @Override
        public String toString() { return name + "(" + cgpa + ")"; }
    }

    public static void main(String[] args) {
        TreeSet<Student> students = new TreeSet<>();
        students.add(new Student("Aman", 8.5));
        students.add(new Student("Priya", 9.2));
        students.add(new Student("Raj", 8.0));
        students.add(new Student("Neha", 9.2));

        for (Student s : students) System.out.println(s);
    }
}
```

Explanation: Implementing Comparable<Student> is required for storing custom objects in a TreeSet — without it, the tree has no basis for determining relative ordering between elements.

compareTo()'s return value convention: negative means “this” precedes the argument, positive means it follows, zero means equality.

Important caveat: TreeSet rejects elements considered duplicates based on compareTo() == 0. If two students share identical CGPA and name, they are treated as duplicates and only one is retained — this is governed by compareTo(), distinct from the separate equals()/hashCode() contract.

Problem 87 — Happy Number (Cycle Detection)

Statement: Determine if a number N is a happy number. Sum of squares of digits, repeated. If reaches 1 → happy. Detect cycle using HashSet.

```

import java.util.HashSet;
import java.util.Set;

public class Solution87 {
    public static void main(String[] args) {
        System.out.println(isHappy(19)); // true
        System.out.println(isHappy(2));  // false
    }

    static boolean isHappy(int n) {
        Set<Integer> seen = new HashSet<>();

        while (n != 1) {
            if (seen.contains(n)) return false;
            seen.add(n);
            n = sumOfSquares(n);
        }

        return true;
    }

    static int sumOfSquares(int n) {
        int sum = 0;
        while (n > 0) {
            int digit = n % 10;
            sum += digit * digit;
            n /= 10;
        }
        return sum;
    }
}

```

Explanation: Key insight: if a number is not happy, it will never reach 1 — instead, it enters a **cycle** indefinitely. The seen set tracks every number encountered during the process; if a number reappears, a cycle is confirmed, and false is returned to avoid an infinite loop.

This is an alternative to Floyd's Cycle Detection (slow/fast pointer technique) — simpler to implement, but trading $O(1)$ space for $O(n)$ space.

Problem 88 — Word Ladder: Shortest Path Length (BFS)

Statement: Given a beginWord, an endWord, and a word list, find the number of steps in the shortest transformation sequence where each step changes exactly one letter. Use BFS and HashSet.

```

import java.util.*;

public class Solution88 {
    public static void main(String[] args) {
        List<String> wordList =
Arrays.asList("hot", "dot", "dog", "lot", "log", "cog");
        System.out.println(wordLadder("hit", "cog", wordList)); //
5
    }

    static int wordLadder(String begin, String end, List<String>

```

```

wordList) {
    Set<String> wordSet = new HashSet<>(wordList);
    if (!wordSet.contains(end)) return 0;

    Queue<String> queue = new LinkedList<>();
    queue.offer(begin);
    Set<String> visited = new HashSet<>();
    visited.add(begin);
    int steps = 1;

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            String word = queue.poll();
            if (word.equals(end)) return steps;

            char[] chars = word.toCharArray();
            for (int j = 0; j < chars.length; j++) {
                char original = chars[j];
                for (char c = 'a'; c <= 'z'; c++) {
                    chars[j] = c;
                    String newWord = new String(chars);
                    if (wordSet.contains(newWord) &&
!visited.contains(newWord)) {
                        visited.add(newWord);
                        queue.offer(newWord);
                    }
                }
                chars[j] = original;
            }
            steps++;
        }
    }

    return 0;
}
}

```

Explanation: This combines two HashSet roles: wordSet validates candidate words in O(1), while visited prevents redundant reprocessing.

At every character position of every word, all 26 letters are tried; any resulting word present in wordSet and not yet visited advances to the next BFS level. BFS proceeds level-by-level, with steps tracking transformation count — upon reaching end, the current step count is the answer.

Problem 89 — Number of Islands (BFS with Visited Set)

Statement: Given a 2D grid of '1' (land) and '0' (water), count islands. Use BFS or DFS with a HashSet of visited cells.

```

import java.util.*;

public class Solution89 {
    public static void main(String[] args) {
        char[][] grid = {

```

```

        {'1','1','0','0'},
        {'1','1','0','0'},
        {'0','0','1','0'},
        {'0','0','0','1'}
    };
    System.out.println(numIslands(grid));    // 3
}

static int numIslands(char[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Set<String> visited = new HashSet<>();
    int count = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            String key = r + "," + c;
            if (grid[r][c] == '1' && !visited.contains(key)) {
                bfs(grid, r, c, visited);
                count++;
            }
        }
    }

    return count;
}

static void bfs(char[][] grid, int row, int col, Set<String>
visited) {
    Queue<int[]> queue = new LinkedList<>();
    queue.offer(new int[]{row, col});
    visited.add(row + "," + col);

    int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};

    while (!queue.isEmpty()) {
        int[] curr = queue.poll();
        int r = curr[0], c = curr[1];

        for (int[] dir : dirs) {
            int nr = r + dir[0], nc = c + dir[1];
            String key = nr + "," + nc;
            if (nr >= 0 && nr < grid.length && nc >= 0 && nc <
grid[0].length &&
                grid[nr][nc] == '1' && !visited.contains(key)) {
                visited.add(key);
                queue.offer(new int[]{nr, nc});
            }
        }
    }
}
}

```

Explanation: The **string-encoding technique** (row + "," + col) produces a unique identifier for each 2D coordinate, suitable for storage in a HashSet<String> — Java arrays cannot be directly used as set elements without custom equals()/hashCode() implementations.

Each time an unvisited land cell is found, a new BFS traversal marks the entire connected island as visited, after which the island counter increments.

Production note: a more performant alternative encodes coordinates as `row * cols + col` into a `Set<Integer>`, avoiding the overhead of string concatenation.

Problem 90 — Reconstruct Itinerary (DFS with PriorityQueue)

Statement: Given airline tickets as from-to pairs, reconstruct the itinerary starting from JFK that visits all airports. Use `HashMap` of airport to `PriorityQueue` for lexical order.

```
import java.util.*;

public class Solution90 {
    public static void main(String[] args) {
        List<List<String>> tickets = Arrays.asList(
            Arrays.asList("JFK", "MUC"), Arrays.asList("MUC", "LHR"),
            Arrays.asList("JFK", "SFO"), Arrays.asList("SFO", "ATL")
        );
        System.out.println(findItinerary(tickets));
    }

    static List<String> findItinerary(List<List<String>> tickets) {
        Map<String, PriorityQueue<String>> graph = new HashMap<>();

        for (List<String> ticket : tickets) {
            graph.computeIfAbsent(ticket.get(0), k -> new
PriorityQueue<>()).offer(ticket.get(1));
        }

        List<String> result = new ArrayList<>();
        dfs("JFK", graph, result);
        Collections.reverse(result);
        return result;
    }

    static void dfs(String airport, Map<String,
PriorityQueue<String>> graph, List<String> result) {
        PriorityQueue<String> destinations = graph.get(airport);
        while (destinations != null && !destinations.isEmpty()) {
            dfs(destinations.poll(), graph, result);
        }
        result.add(airport);
    }
}
```

Explanation: This is an Eulerian Path problem, an advanced graph theory concept. The `PriorityQueue<String>` maintains each airport's destinations in **lexicographically sorted** order — ensuring that, when multiple choices exist, the alphabetically smallest is always attempted first, producing the correct lexicographically smallest itinerary.

DFS recursively exhausts all available tickets before adding the current airport to the result — a **post-order** DFS structure.

The final reversal is required because airports are added to the result in the order they become “dead ends” (the last airport reached is added first), which is the reverse of the actual travel sequence.

Problem 91 — Design Twitter (Map + Set + PriorityQueue)

Statement: Design a simplified Twitter with postTweet, getNewsFeed, follow, and unfollow. Use HashMap of userId to a Set of followees and a PriorityQueue for the feed.

```
import java.util.*;

public class Solution91 {
    static class Twitter {
        Map<Integer, List<int[]>> tweets = new HashMap<>();
        Map<Integer, Set<Integer>> follows = new HashMap<>();
        int time = 0;

        void postTweet(int userId, int tweetId) {
            tweets.computeIfAbsent(userId, k -> new ArrayList<>
            (())).add(new int[]{time--, tweetId});
        }

        List<Integer> getNewsFeed(int userId) {
            PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -
            > a[0] - b[0]);

            Set<Integer> sources = new HashSet<>
            (follows.getDefault(userId, new HashSet<>()));
            sources.add(userId);

            for (int uid : sources) {
                List<int[]> userTweets = tweets.getDefault(uid,
                new ArrayList<>());

                int start = Math.max(0, userTweets.size() - 10);
                for (int i = start; i < userTweets.size(); i++) {
                    heap.offer(userTweets.get(i));
                }
            }

            List<Integer> result = new ArrayList<>();
            while (!heap.isEmpty() && result.size() < 10) {
                result.add(heap.poll()[1]);
            }
            return result;
        }

        void follow(int followerId, int followeeId) {
            follows.computeIfAbsent(followerId, k -> new HashSet<>
            (())).add(followeeId);
        }

        void unfollow(int followerId, int followeeId) {
            Set<Integer> set = follows.get(followerId);
            if (set != null) set.remove(followeeId);
        }
    }

    public static void main(String[] args) {
        Twitter tw = new Twitter();
        tw.postTweet(1, 5);
        tw.follow(1, 2);
    }
}
```

```

        tw.postTweet(2, 6);
        System.out.println(tw.getNewsFeed(1));    // [6, 5]
    }
}

```

Explanation: This is a realistic system-design exercise combining multiple data structures. `Map<Integer, Set<Integer>>` represents follow relationships — a `Set` is used here because “already following” checks must be $O(1)$, and duplicate follows are disallowed by definition.

The decrementing time-- counter is a deliberate technique: it ensures more recent tweets receive a **smaller** time value, which the min-heap then naturally prioritizes (recent tweets surface first).

`getNewsFeed()` collects tweets from all relevant sources (the user plus their followees) into a single heap, then extracts the top 10 most recent.

Problem 92 — Longest Substring with At Most K Distinct Characters

Statement: Given a string, find the length of the longest substring that contains at most K distinct characters. Use `HashSet` or `HashMap` with a sliding window.

```

import java.util.HashMap;
import java.util.Map;

public class Solution92 {
    public static void main(String[] args) {
        System.out.println(longestKDistinct("eceba", 2));    // 3
    }

    static int longestKDistinct(String s, int k) {
        Map<Character, Integer> freq = new HashMap<>();
        int left = 0, best = 0;

        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);

            while (freq.size() > k) {
                char leftChar = s.charAt(left);
                freq.put(leftChar, freq.get(leftChar) - 1);
                if (freq.get(leftChar) == 0) freq.remove(leftChar);
                left++;
            }

            best = Math.max(best, right - left + 1);
        }

        return best;
    }
}

```

Explanation: This combines a sliding window with a frequency map. `freq.size()` indicates how many **distinct** characters are currently in the window — a `Set`-like property naturally provided by the map’s key collection.

When distinct character count exceeds k , the window contracts from the left until the constraint is satisfied again.

Important: when a character's frequency reaches zero, it must be explicitly **removed** from the map — leaving it at zero would corrupt the `freq.size()` count, since the key would still be present.

Problem 93 — Divide into K Consecutive Groups

Statement: Given a list of integers, check if the list can be divided into groups of size K where each group consists of K consecutive integers. Use `TreeMap` to track remaining counts.

```
import java.util.*;

public class Solution93 {
    public static void main(String[] args) {
        int[] nums = {1,2,3,3,4,4,5,6};
        System.out.println(canDivide(nums, 4)); // true
    }

    static boolean canDivide(int[] nums, int k) {
        TreeMap<Integer, Integer> freq = new TreeMap<>();
        for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);

        while (!freq.isEmpty()) {
            int start = freq.firstKey();
            int count = freq.get(start);

            for (int i = 0; i < k; i++) {
                int curr = start + i;
                if (!freq.containsKey(curr) || freq.get(curr) <
count) return false;

                int newCount = freq.get(curr) - count;
                if (newCount == 0) freq.remove(curr);
                else freq.put(curr, newCount);
            }
        }

        return true;
    }
}
```

Explanation: `TreeMap` is used because the **smallest key** must be repeatedly retrieved (`firstKey()`, $O(\log n)$) — a natural advantage of this sorted structure.

The greedy approach: always begin a group with the smallest available number, then require exactly K consecutive numbers with sufficient remaining frequency to match the starting number's count.

If, at any point, a required consecutive number is missing or has insufficient frequency, the division is impossible, and the method returns false.

Problem 94 — K Closest Points to the Origin

Statement: Given a list of points, find the K closest points to the origin. Use a max-heap PriorityQueue of size K storing distances.

```
import java.util.*;

public class Solution94 {
    public static void main(String[] args) {
        int[][] points = {{1,3},{-2,2},{5,8},{0,1}};
        int[][] result = kClosest(points, 2);
        for (int[] p : result)
            System.out.println(Arrays.toString(p));
    }

    static int[][] kClosest(int[][] points, int k) {
        PriorityQueue<int[]> maxHeap = new PriorityQueue<>(
            (a, b) -> (b[0]*b[0] + b[1]*b[1]) - (a[0]*a[0] +
a[1]*a[1])
        );

        for (int[] point : points) {
            maxHeap.offer(point);
            if (maxHeap.size() > k) maxHeap.poll();
        }

        return maxHeap.toArray(new int[0][]);
    }
}
```

Explanation: This follows the **max-heap of size k** pattern, the inverse of the Kth-Largest technique (Problem 65). The heap always retains “the k closest points found so far,” and the point with the **greatest** distance occupies the heap’s top — when a closer point is found, the farthest is discarded.

The distance formula $x^2 + y^2$ is used directly; taking the square root is unnecessary for comparison purposes, since it is a monotonic transformation that preserves relative ordering.

Problem 95 — Cycle Detection in a Directed Graph (DFS with Recursion Stack)

Statement: Given a directed graph represented as a HashMap of node to list of neighbors, detect whether a cycle exists using DFS with a visited Set and a recursion stack Set.

```
import java.util.*;

public class Solution95 {
    public static void main(String[] args) {
        Map<Integer, List<Integer>> graph = new HashMap<>();
        graph.put(0, Arrays.asList(1));
        graph.put(1, Arrays.asList(2));
        graph.put(2, Arrays.asList(3));
        graph.put(3, Arrays.asList(1));    // cycle: 1->2->3->1

        System.out.println(hasCycle(graph, 4));    // true
    }

    static boolean hasCycle(Map<Integer, List<Integer>> graph, int
```

```

n) {
    Set<Integer> visited = new HashSet<>();
    Set<Integer> recStack = new HashSet<>();

    for (int i = 0; i < n; i++) {
        if (!visited.contains(i)) {
            if (dfs(i, graph, visited, recStack)) return true;
        }
    }
    return false;
}

static boolean dfs(int node, Map<Integer, List<Integer>> graph,
    Set<Integer> visited, Set<Integer> recStack)
{
    visited.add(node);
    recStack.add(node);

    for (int neighbor : graph.getOrDefault(node, new ArrayList<>
    ())) {
        if (!visited.contains(neighbor)) {
            if (dfs(neighbor, graph, visited, recStack)) return
true;
        } else if (recStack.contains(neighbor)) {
            return true;
        }
    }

    recStack.remove(node);
    return false;
}
}

```

Explanation: This uses two distinct Set instances, each serving a different purpose. `visited` globally tracks all explored nodes and is never cleared. `recStack` tracks only the **current DFS path** — added when a node begins exploration, removed when DFS backtracks away from that node.

The core insight for cycle detection: if a neighbor exists in `recStack`, the search has encountered a “back edge” — confirming a cycle, since that neighbor lies somewhere earlier in the current active path.

Distinction from undirected graphs: undirected cycle detection only requires checking against the immediate parent. Directed graphs require tracking the entire recursion stack, since edges are one-directional and a node could be revisited through a different, non-cyclic path.

Problem 96 — Longest Word Buildable One Character at a Time

Statement: Given a list of words, find the longest word that can be built one character at a time by other words in the list. Each prefix must also exist in the list. Use a HashSet.

```
import java.util.*;
```

```

public class Solution96 {
    public static void main(String[] args) {
        String[] words =
{"a", "banana", "app", "appl", "ap", "apply", "apple"};
        System.out.println(longestWord(words));    // apple
    }

    static String longestWord(String[] words) {
        Set<String> wordSet = new HashSet<>(Arrays.asList(words));
        Arrays.sort(words);

        String best = "";

        for (String word : words) {
            boolean valid = true;
            for (int i = 1; i < word.length(); i++) {
                if (!wordSet.contains(word.substring(0, i))) {
                    valid = false;
                    break;
                }
            }
            if (valid && word.length() > best.length()) {
                best = word;
            }
        }

        return best;
    }
}

```

Explanation: wordSet provides O(1) checks for “does this prefix exist in the word list?” For each candidate word, every prefix is checked (word.substring(0, i) for each i) — if any prefix is missing, the word is not “buildable.”

Sorting technique: sorting words alphabetically before iteration ensures that, among multiple valid words of equal length, the alphabetically smaller one is selected automatically — since the comparison uses strict >, equal-length ties do not overwrite the existing best.

Problem 97 — Four Sum

Statement: Given a list of integers, find all unique quadruplets that sum to a given target. Use sorting combined with a HashSet to avoid duplicates efficiently.

```

import java.util.*;

public class Solution97 {
    public static void main(String[] args) {
        int[] nums = {1, 0, -1, 0, -2, 2};
        System.out.println(fourSum(nums, 0));
    }

    static List<List<Integer>> fourSum(int[] nums, int target) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        int n = nums.length;

```

```

        for (int i = 0; i < n - 3; i++) {
            if (i > 0 && nums[i] == nums[i-1]) continue;

            for (int j = i + 1; j < n - 2; j++) {
                if (j > i + 1 && nums[j] == nums[j-1]) continue;

                int left = j + 1, right = n - 1;
                while (left < right) {
                    long sum = (long) nums[i] + nums[j] + nums[left]
+ nums[right];

                    if (sum == target) {
                        result.add(Arrays.asList(nums[i], nums[j],
nums[left], nums[right]));
                        while (left < right && nums[left] ==
nums[left+1]) left++;
                        while (left < right && nums[right] ==
nums[right-1]) right--;
                        left++; right--;
                    } else if (sum < target) left++;
                    else right--;
                }
            }
        }

        return result;
    }
}

```

Explanation: This extends the Three Sum approach (Problem 20) with an additional nested loop, since four numbers must now be fixed in combination — two via nested loops, two via two-pointer search.

`long sum` prevents integer overflow, since the sum of four large integers can exceed `int` range.

Duplicate skipping is required at **multiple nesting levels** — both outer loops (`i` and `j`) and the inner two-pointer search — reflecting the increased combinatorial complexity relative to Three Sum.

Problem 98 — Accounts Merge (Union-Find with Email Mapping)

Statement: Given accounts with emails, merge accounts that share any email using Union-Find with a `HashMap` mapping each email to its owner account index.

```

import java.util.*;

public class Solution98 {
    static Map<String, String> parent = new HashMap<>();

    public static void main(String[] args) {
        List<List<String>> accounts = Arrays.asList(
            Arrays.asList("Aman", "aman@g.com", "aman@fb.com"),
            Arrays.asList("Priya", "priya@g.com"),
            Arrays.asList("Aman", "aman@fb.com", "aman@tw.com")
        );
    }
}

```



```

        for (List<String> account : accountsMerge(accounts)) {
            System.out.println(account);
        }
    }

    static String find(String x) {
        parent.putIfAbsent(x, x);
        if (!parent.get(x).equals(x)) {
            parent.put(x, find(parent.get(x)));
        }
        return parent.get(x);
    }

    static void union(String x, String y) {
        parent.put(find(x), find(y));
    }

    static List<List<String>> accountsMerge(List<List<String>>
accounts) {
        Map<String, String> emailToName = new HashMap<>();

        for (List<String> account : accounts) {
            String name = account.get(0);
            for (int i = 1; i < account.size(); i++) {
                emailToName.put(account.get(i), name);
                union(account.get(1), account.get(i));
            }
        }

        Map<String, List<String>> groups = new TreeMap<>();
        for (String email : emailToName.keySet()) {
            groups.computeIfAbsent(find(email), k -> new ArrayList<>
()).add(email);
        }

        List<List<String>> result = new ArrayList<>();
        for (List<String> emails : groups.values()) {
            Collections.sort(emails);
            List<String> account = new ArrayList<>();
            account.add(emailToName.get(emails.get(0)));
            account.addAll(emails);
            result.add(account);
        }

        return result;
    }
}

```

Explanation: This is a practical application of Union-Find (Disjoint Set Union). The parent map tracks each email's "root representative" — initially, each email is its own parent.

`find()` traverses to the root using **path compression**, directly attaching each visited node to the root to accelerate future lookups.

`union()` merges two emails' groups by connecting their respective roots.

To produce final groupings, each email's `find()` result is computed; emails sharing the same root belong to the same merged account.

Problem 99 — All Distinct Palindromic Substrings

Statement: Given a string, find all distinct palindromic substrings. Use a set to store results and expand around each center for $O(n^2)$ solution.

```
import java.util.*;

public class Solution99 {
    public static void main(String[] args) {
        System.out.println(palindromicSubstrings("aabaa"));
    }

    static List<String> palindromicSubstrings(String s) {
        Set<String> result = new HashSet<>();
        int n = s.length();

        for (int i = 0; i < n; i++) {
            expand(s, i, i, result);
            expand(s, i, i + 1, result);
        }

        List<String> sorted = new ArrayList<>(result);
        sorted.sort((a, b) -> a.length() - b.length());
        return sorted;
    }

    static void expand(String s, int left, int right, Set<String>
result) {
        while (left >= 0 && right < s.length() && s.charAt(left) ==
s.charAt(right)) {
            result.add(s.substring(left, right + 1));
            left--;
            right++;
        }
    }
}
```

Explanation: This uses the **expand-around-center** technique. Every palindrome has a center — odd-length palindromes center on a single character; even-length palindromes center between two adjacent characters.

result, declared as HashSet<String>, automatically eliminates duplicate palindromic substrings (e.g., "aa" may be discoverable from multiple centers, but the Set retains only one instance).

Expansion from each center continues as long as boundary characters match; the process halts at the first mismatch.

Problem 100 — Distinct Subsequences Count

Statement: Given a string, count the number of distinct subsequences that equal a given target string.

```
public class Solution100 {
    public static void main(String[] args) {
        System.out.println(numDistinct("rabbbit", "rabbit"));    // 3
    }
}
```

```

    }

    static int numDistinct(String s, String t) {
        int m = s.length(), n = t.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) dp[i][0] = 1;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                dp[i][j] = dp[i-1][j];
                if (s.charAt(i-1) == t.charAt(j-1)) {
                    dp[i][j] += dp[i-1][j-1];
                }
            }
        }

        return dp[m][n];
    }
}

```

Explanation: This is a dynamic programming problem, grouped here for its conceptual relationship to subsequence-uniqueness topics. $dp[i][j]$ represents “the number of ways to form $t[0..j-1]$ as a subsequence of $s[0..i-1]$.”

For each character pair, two choices exist: **skip** the current character of s (inherit $dp[i-1][j]$), or **use** it if it matches the current target character (additionally inherit $dp[i-1][j-1]$).

Base case: $dp[i][0] = 1$ — an empty target string is always satisfied trivially (by selecting nothing), regardless of how much of s has been considered.

This is structurally similar to the dynamic programming patterns used in Edit Distance and Longest Common Subsequence — a 2D table with character-by-character comparison decisions.

End of Module 4 — Set

Document Summary

Module	Problems	Count
List	1-24	24
Map	25-50	26
Queue	51-74	24
Set	75-100	26
Total		100

Repository: SuperFaculties — DSA Training Materials **Prepared by:** Nidhish Raj Mourya | +91-6265883747